Mooc 2 module 5-7

Prompt sử dụng AI copilot trong lúc học: Bạn là một giảng viên giống như giảng viên của khóa học này coursera là một chuyên gia trong lĩnh vực SE modeling và SE UML hãy giảng cho tôi về những gì đang hiển thị và chương tôi đang học

🎯 Mục tiêu của chương này

- Hiểu **Use Case Modeling** là gì và tại sao nó quan trọng.
- Biết cách xác định **actors** trong một hệ thống phần mềm.
- Phân biệt actor với các thành phần khác (ví dụ domain classes, thiết bị phần cứng).
- Thực hành nhận diện actors từ một bài toán thực tế.

📌 Nội dung chính

1. Use Case Modeling

- Dùng để **mô tả hành vi của hệ thống từ góc nhìn người dùng**.
- Giúp:
    - Thu thập yêu cầu chức năng.
    - Lập kế hoạch phát triển theo từng iteration.
    - Xác nhận hệ thống có đáp ứng đúng yêu cầu hay không.
- Use cases chính là **động lực dẫn dắt quá trình phát triển phần mềm**.

2. Actor là gì?

- **Actor**: một thực thể bên ngoài hệ thống, có thể là:
    - Người dùng (ví dụ: sinh viên, giảng viên).
    - Hệ thống khác (ví dụ: hệ thống thanh toán).
- Actor **cung cấp đầu vào hoặc nhận đầu ra từ hệ thống**.
- Actor là một **stereotype của UML class** (tức là một loại lớp đặc biệt trong UML).

3. Cách xác định Actor

Bạn có thể tự hỏi:

- Ai sẽ sử dụng hệ thống?
- Họ đóng vai trò gì trong tương tác?
- Ai cung cấp hoặc nhận thông tin từ hệ thống?
- Có hệ thống nào khác tương tác với hệ thống này không?
- Ai cài đặt, khởi động, bảo trì hệ thống?

⚠️ Lưu ý: **thiết bị phần cứng** như bàn phím, chuột, máy in **không phải là actor**.

4. Ví dụ minh họa

Giả sử bạn xây dựng **hệ thống đăng ký môn học cho một trường đại học (ASU)**:

- **Student**: đăng ký môn học.
- **Instructor**: giảng viên quản lý lớp học.
- **Billing System**: hệ thống bên ngoài phụ trách thu học phí.

Trong **Use Case Diagram**:

- Bạn vẽ một hình chữ nhật đại diện cho hệ thống.
- Các actors (Student, Instructor, Billing System) nằm bên ngoài.
- Các use cases (Register Course, Pay Tuition, Assign Instructor…) nằm bên trong hệ thống.
- Actor kết nối với use case bằng đường thẳng.

✨ Tóm lại

- Actor là **cửa ngõ** để khám phá các use case.
- Một người dùng có thể đóng nhiều vai trò, và một vai trò có thể được nhiều người dùng đảm nhận.
- Actor giúp bạn hình dung rõ ràng **ai tương tác với hệ thống và vì mục đích gì**.

🎯 Mục tiêu của bài học

- Hiểu khái niệm **use case** trong UML.
- Biết cách mô tả hành vi hệ thống thông qua use case.
- Phân biệt use case với các chức năng nội bộ.
- Thực hành xây dựng sơ đồ use case từ yêu cầu thực tế.

📌 Nội dung chính

1. Use Case là gì?

- **Use case**: mô tả một **tập hợp các hành động** mà hệ thống thực hiện để mang lại kết quả có giá trị cho một actor.
- Nó không mô tả chi tiết cách hệ thống thực hiện, mà chỉ tập trung vào **hành vi quan sát được từ bên ngoài**.
- Mỗi use case thường được viết dưới dạng **kịch bản** (scenario).

Ví dụ:

Trong hệ thống đăng ký môn học, một use case có thể là **Register Course** – sinh viên chọn môn học và hệ thống xác nhận đăng ký.

2. Đặc điểm của Use Case

- **Có giá trị cho actor**: kết quả phải hữu ích cho người dùng hoặc hệ thống bên ngoài.
- **Được kích hoạt bởi actor**: actor khởi động use case bằng một hành động.
- **Kết thúc khi đạt mục tiêu**: use case có điểm bắt đầu và kết thúc rõ ràng.

3. Các loại Use Case

- **Primary use case**: mô tả chức năng chính mà actor mong muốn (ví dụ: đăng ký môn học).
- **Secondary use case**: hỗ trợ hoặc bổ sung cho primary (ví dụ: hủy đăng ký, xem lịch học).

4. Biểu diễn trong UML

- Trong sơ đồ use case:
  - Use case được vẽ bằng **hình elip** bên trong hệ thống.
  - Actor kết nối với use case bằng đường thẳng.
- Một actor có thể tham gia nhiều use case, và một use case có thể liên quan đến nhiều actor.

5. Ví dụ minh họa

Hệ thống đăng ký môn học (ASU):

- **Actors**: Student, Instructor, Billing System.
- **Use cases**:
  - Register Course (Student ↔ System).
  - Assign Instructor (Instructor ↔ System).
  - Pay Tuition (Billing System ↔ System).

Sơ đồ sẽ có các elip tương ứng với từng use case, nối với actors bên ngoài.

✨ Tóm lại

- Nếu **actors** là “ai”, thì **use cases** là “họ làm gì”.
- Use case giúp bạn mô tả **yêu cầu chức năng** của hệ thống một cách trực quan.
- Đây là bước nền tảng để sau này bạn viết **use case specification** chi tiết (bao gồm các bước, điều kiện, kết quả).

📌 Các bước chính trong ví dụ

1. Phân tích yêu cầu từng câu

Đọc **prompt statement** (mô tả bài toán) từng câu để tìm ra chức năng.

- Ví dụ:
- Sinh viên yêu cầu xem **course catalog** → chức năng *Browse Course Offerings*.
- Ai đó chuẩn bị course catalog → chức năng *Prepare Course Offerings*.

- Sinh viên chọn môn học, thêm, hủy, đổi lịch → các chức năng liên quan đến *Register for Courses*.
- Instructor chọn môn để dạy, yêu cầu danh sách sinh viên → chức năng *Select Courses to Teach*, *Request Enrollment List*.
- Billing System nhận thông tin học phí → chức năng *Receive Billing Information*.
- Registrar quản lý thông tin môn học → chức năng *Maintain Course Information*.

2. Nhận diện actors

Từ các chức năng trên, ta thấy có các actors:

- **Student**
- **Instructor**
- **Billing System**
- **Registrar**

3. Gom nhóm chức năng thành use case hoàn chỉnh

- Nếu ta vẽ tất cả chức năng riêng lẻ, sơ đồ sẽ có quá nhiều elip, khó đọc.
- Thay vào đó, ta **gom nhóm các chức năng nhỏ thành một use case lớn**:
    - *Register for Courses* (bao gồm browse, add, drop, change schedule).
    - *Maintain Course Information* (bao gồm prepare, cancel course offerings).
    - *Select Courses to Teach* và *Request Enrollment List* giữ riêng cho Instructor.

👉 Quy tắc: **Một use case tốt phải là một câu chuyện hoàn chỉnh từ đầu đến cuối**, không chỉ là mảnh vụn.

4. Vẽ sơ đồ Use Case

- Student ↔ Register for Courses.
- Instructor ↔ Select Courses to Teach, Request Enrollment List.
- Registrar ↔ Maintain Course Information.
- Billing System ↔ liên kết với Register for Courses (nhận thông tin học phí).

Lưu ý:

- Mũi tên từ Student đến use case cho thấy Student **khởi động** use case.
- Đường nối với Billing System không có mũi tên, vì Billing System chỉ **nhận thông tin**, không khởi động use case.

5. Tổng kết

- Bài học này nhấn mạnh: **đừng để sơ đồ quá chi tiết với nhiều use case nhỏ**.
- Hãy nhóm các chức năng thành **use case lớn, hoàn chỉnh** để sơ đồ dễ đọc, dễ hiểu.
- Sau khi có sơ đồ, bước tiếp theo là viết **Use Case Specification** cho từng use case chính.

📌 Các bước chính trong ví dụ

1. Phân tích yêu cầu từng câu

Đọc **prompt statement** (mô tả bài toán) từng câu để tìm ra chức năng.

- Ví dụ:
- Sinh viên yêu cầu xem **course catalog** → chức năng *Browse Course Offerings*.
- Ai đó chuẩn bị course catalog → chức năng *Prepare Course Offerings*.
- Sinh viên chọn môn học, thêm, hủy, đổi lịch → các chức năng liên quan đến *Register for Courses*.
- Instructor chọn môn để dạy, yêu cầu danh sách sinh viên → chức năng *Select Courses to Teach*, *Request Enrollment List*.
- Billing System nhận thông tin học phí → chức năng *Receive Billing Information*.
- Registrar quản lý thông tin môn học → chức năng *Maintain Course Information*.

2. Nhận diện actors

Từ các chức năng trên, ta thấy có các actors:

- **Student**
- **Instructor**
- **Billing System**
- **Registrar**

3. Gom nhóm chức năng thành use case hoàn chỉnh

- Nếu ta vẽ tất cả chức năng riêng lẻ, sơ đồ sẽ có quá nhiều elip, khó đọc.
- Thay vào đó, ta **gom nhóm các chức năng nhỏ thành một use case lớn**:
    - *Register for Courses* (bao gồm browse, add, drop, change schedule).
    - *Maintain Course Information* (bao gồm prepare, cancel course offerings).
    - *Select Courses to Teach* và *Request Enrollment List* giữ riêng cho Instructor.

👉 Quy tắc: **Một use case tốt phải là một câu chuyện hoàn chỉnh từ đầu đến cuối**, không chỉ là mảnh vụn.

4. Vẽ sơ đồ Use Case

- Student ↔ Register for Courses.
- Instructor ↔ Select Courses to Teach, Request Enrollment List.
- Registrar ↔ Maintain Course Information.
- Billing System ↔ liên kết với Register for Courses (nhận thông tin học phí).

Lưu ý:

- Mũi tên từ Student đến use case cho thấy Student **khởi động** use case.
- Đường nối với Billing System không có mũi tên, vì Billing System chỉ **nhận thông tin**, không khởi động use case.

5. Tổng kết

- Bài học này nhấn mạnh: **đừng để sơ đồ quá chi tiết với nhiều use case nhỏ**.
- Hãy nhóm các chức năng thành **use case lớn, hoàn chỉnh** để sơ đồ dễ đọc, dễ hiểu.
- Sau khi có sơ đồ, bước tiếp theo là viết **Use Case Specification** cho từng use case chính.

🎯 Mục tiêu

- Hiểu cấu trúc của một **Use Case Specification**.
- Biết cách viết **preconditions, postconditions, flow of events**.
- Thấy ví dụ thực tế (ATM, hệ thống đăng ký môn học).

📌 Nội dung chính

1. Thành phần của Use Case Specification

Một tài liệu use case thường gồm:

- **Tên use case**: ví dụ *Withdraw Money*.
- **Mô tả ngắn gọn**: mục tiêu của use case.
- **Actors tham gia**: ai khởi động và ai liên quan.
- **Preconditions**: điều kiện phải đúng trước khi thực hiện.
- **Flow of Events**: luồng sự kiện chính (basic flow).
- **Alternative Flows**: các luồng phụ, ngoại lệ, tình huống đặc biệt.
- **Postconditions**: trạng thái hệ thống sau khi hoàn tất.
- **Non-functional requirements** (nếu có): ví dụ tốc độ phản hồi, bảo mật.

2. Ví dụ ATM – Withdraw Money

- **Tên**: Withdraw Money.
- **Actor**: Customer.
- **Precondition**: Tài khoản phải có ≥ 100.
- **Basic Flow**:
    - Khách hàng chọn “Withdraw Money”.
    - Hệ thống yêu cầu nhập số tiền.
    - Khách hàng nhập số tiền.
    - Hệ thống kiểm tra số dư.
    - Nếu hợp lệ, hệ thống xuất tiền và cập nhật số dư.
- **Alternative Flow**:
    - Nếu số dư < số tiền yêu cầu → hiển thị thông báo lỗi.
- **Postcondition**: Số dư tài khoản ≥ 0.

3. Ví dụ hệ thống đăng ký môn học (ASU)

Use case: **Select Courses to Teach**

- **Actor**: Instructor.
- **Precondition**: Học kỳ chưa bắt đầu, hạn đăng ký chưa hết.
- **Basic Flow**:
    - Instructor chọn chức năng “Select Courses to Teach”.
    - Hệ thống hiển thị danh sách môn học khả dụng.
    - Instructor chọn môn muốn dạy.
    - Hệ thống xác nhận và lưu thông tin.
- **Alternative Flow**:
    - Nếu môn học đã bị hủy hoặc hết hạn đăng ký → thông báo lỗi.
- **Postcondition**: Instructor được gán vào môn học.

✨ Tóm lại

- **Use Case Specification** là cầu nối từ sơ đồ UML sang tài liệu chi tiết.
- Nó giúp đảm bảo tất cả stakeholders hiểu rõ hành vi hệ thống.
- Luôn viết **basic flow trước**, sau đó thêm **alternative flows**.

📌 Extension Point

- **Extension Point**: là một **nhãn trong luồng sự kiện** nơi có thể chèn thêm hành vi bổ sung.
- Dùng để đánh dấu vị trí mà **alternative flow** có thể xảy ra.
- Có 3 loại:
    1. **Single Location Extension Point**: xuất hiện một lần duy nhất trong use case.
    2. **Discrete Locations Extension Point**: xuất hiện nhiều lần tại các vị trí khác nhau.
    3. **Region Extension Point**: xác định một vùng (từ điểm bắt đầu đến điểm kết thúc) trong luồng sự kiện, nơi có thể chèn hành vi đặc biệt.

Ví dụ: Trong use case *Modify Schedule*, ta có thể định nghĩa một **region extension point** từ “Begin Modifying Schedule” đến “Confirm Modifying Schedule”.

📌 Alternative Flow

- **Alternative Flow**: mô tả **hành vi tùy chọn, biến thể hoặc ngoại lệ** so với luồng chính.
- Được đánh số A1, A2, … để dễ theo dõi.
- Có 3 loại:
    1. **Specific Alternative Flow**: bắt đầu tại một extension point cụ thể.
    2. **Bounded Alternative Flow**: xảy ra trong phạm vi giữa hai extension points.
    3. **General Alternative Flow**: có thể xảy ra ở bất kỳ điểm nào trong luồng sự kiện.

📌 Ví dụ minh họa

- **Exceptional Flow**: Instructor nhập sai năm học → hệ thống báo lỗi và yêu cầu nhập lại.
- **Variant Flow**: Nếu lịch học đã tồn tại, hệ thống hiển thị thông báo và chuyển hướng đến extension point khác.
- **Optional Flow**: Một chức năng hiếm khi dùng, ví dụ hiển thị thêm thông tin phụ, có thể được thêm vào luồng chính nhưng không bắt buộc.

✨ Tóm lại

- **Extension Points** là "cửa" để chèn thêm hành vi.
- **Alternative Flows** là cách mô tả các tình huống ngoại lệ, biến thể, hoặc tùy chọn.
- Chúng giúp use case specification trở nên **thực tế, đầy đủ và dễ kiểm thử**.

📌 Khái niệm SubFlow

- **Subflow**: là một **đoạn sự kiện có mục đích rõ ràng và mang tính nguyên tử** trong use case.
- Nó không phải là một use case riêng biệt, mà chỉ là **một phần của use case**.
- Mục tiêu: cải thiện **tính dễ đọc** và **tổ chức** của tài liệu use case.

📌 Quy tắc khi dùng SubFlow

- Đặt tên cho subflow (ví dụ: *Create Schedule*, *Modify Schedule*).
- Đánh số subflow: S1, S2, … để dễ tham chiếu.
- **Tránh lồng quá nhiều subflow** (không nên có subflow trong subflow trong subflow), vì sẽ làm tài liệu khó đọc.
- Subflow chỉ nên dùng khi đoạn sự kiện có **ý nghĩa riêng biệt** và có thể tách ra.

📌 Ví dụ minh họa

Trong use case **Register for Courses**:

- **Basic Flow**: Sinh viên đăng ký môn học.
- Thay vì viết tất cả chi tiết trong một luồng dài, ta chia thành subflow:
    - **S1: Create Schedule** – mô tả các bước tạo lịch học.
    - **S2: Modify Schedule** – mô tả các bước chỉnh sửa lịch học.

👉 Nhờ vậy, **basic flow** trở nên ngắn gọn, dễ đọc. Nếu muốn chi tiết, người đọc có thể xem phần subflow.

📌 Lợi ích

- Giúp **stakeholders** (người dùng, tester, developer) dễ hiểu hơn.

- Giảm sự mơ hồ: mỗi subflow mô tả rõ ràng hành vi cụ thể.
- Tránh việc chia nhỏ quá mức thành nhiều use case phụ thuộc lẫn nhau. Thay vào đó, gom chúng thành một use case lớn với các subflow.

Ví dụ: thay vì có các use case riêng lẻ *Select Product*, *Enter Order Info*, *Enter Shipping Info*, *Enter Payment Info*, ta gom lại thành một use case lớn *Place Order*, rồi chia thành các subflow bên trong.

✨ Tóm lại

- **Subflow = đoạn sự kiện có mục đích rõ ràng trong use case**.
- Giúp tài liệu ngắn gọn, dễ đọc, dễ bảo trì.
- Tránh lồng quá nhiều subflow và tránh chia nhỏ thành các use case phụ thuộc.

📌 Non-Functional Requirements (NFRs) là gì?

- **Định nghĩa**: Là những ràng buộc hoặc tiêu chí chất lượng áp đặt lên hệ thống hoặc use case.
- Chúng không mô tả *hệ thống làm gì*, mà mô tả *hệ thống làm như thế nào*.
- Ví dụ: tốc độ phản hồi, độ tin cậy, khả năng bảo trì, bảo mật, giao diện, môi trường vận hành.

📌 Các loại NFR phổ biến

1. **Design Quality**
    - Reliability (độ tin cậy).
    - Maintainability (dễ bảo trì).
    - Supportability (hỗ trợ nâng cấp).
    - Documentation (tài liệu cần thiết).
2. **Hardware Constraints**
    - Nền tảng triển khai.
    - Bộ nhớ, dung lượng lưu trữ.
3. **Implementation Constraints**
    - Ngôn ngữ lập trình, chuẩn coding.
    - Cách xử lý lỗi.
4. **Interface Constraints**
    - Giao diện người dùng.
    - Giao tiếp với hệ thống bên ngoài.
5. **Management Requirements**
    - Backup, cài đặt, bảo trì.
6. **Performance Requirements**
    - Tốc độ, thời gian phản hồi, độ chính xác.
7. **Physical Environment**
    - Điều kiện bất thường, hệ thống phân tán.
8. **Security Requirements**
    - Truy cập hệ thống, dữ liệu, bảo mật vật lý.

📌 Ví dụ minh họa – *Set Watch (đồng hồ vệ tinh)*

- Người dùng chỉ cần biết đọc đồng hồ số → **Usability/Interface**.
- Không có lỗi phần mềm buộc reset → **Reliability**.
- Hỗ trợ nâng cấp qua USB → **Supportability**.
- Hiển thị đúng giờ trong vòng 5 phút sau khi mất tín hiệu GPS → **Response Time**.
- Độ chính xác trong vòng 1/100 giây trong 5 năm → **Accuracy**.
- Hiển thị đúng giờ ở 24 múi giờ → **Performance**.
- Phần mềm viết bằng Java → **Implementation**.
- Tuân thủ chuẩn API 2.0 → **Interface**.

📌 Administration Use Cases

- Một số NFR có thể được mô tả dưới dạng **administration use cases**.
- Ví dụ: *System Startup*, *System Shutdown*, *System Backup*, *Locking*.
- Đây là cách đưa các yêu cầu phi chức năng vào sơ đồ use case để quản lý.

✨ Tóm lại

- **NFRs = ràng buộc chất lượng**.
- Chúng bổ sung cho use case chức năng, đảm bảo hệ thống không chỉ chạy được mà còn chạy tốt.
- Có thể mô tả NFRs trực tiếp trong tài liệu hoặc thông qua **administration use cases**.

📌 Các tiêu chí khi validate yêu cầu

1. **Complete** – đầy đủ
    - Tất cả yêu cầu cần thiết phải được ghi nhận.
    - Không bỏ sót chức năng hoặc ràng buộc nào.
2. **Consistent** – nhất quán
    - Các yêu cầu không được mâu thuẫn với nhau.
    - Ví dụ: một yêu cầu nói “không cần nâng cấp phần mềm”, nhưng yêu cầu khác lại nói “phần mềm phải dễ dàng nâng cấp qua USB” → mâu thuẫn.
3. **Clear** – rõ ràng
    - Mọi stakeholder đều có thể hiểu.
    - Tránh dùng từ mơ hồ, ví dụ “hệ thống phải nhanh” → cần cụ thể hóa thành “thời gian phản hồi ≤ 2 giây”.
4. **Correct** – chính xác
    - Yêu cầu phải phản ánh đúng thực tế.
    - Ví dụ: SatWatch chỉ hỗ trợ 24 múi giờ là sai, vì thực tế có hơn 24 múi giờ.
5. **Realistic/Feasible** – khả thi
    - Yêu cầu phải có thể thực hiện được với công nghệ và nguồn lực hiện tại.

📌 Công cụ validate

- **Acceptance Test**: phương pháp chính để kiểm tra xem hệ thống có đáp ứng yêu cầu hay không.
- Các yêu cầu được kiểm thử trực tiếp trước khách hàng hoặc người dùng.

📌 Ví dụ minh họa (SatWatch)

- **Incomplete**: không mô tả hành vi khi người dùng đứng giữa hai múi giờ → cần thêm yêu cầu "thay đổi múi giờ không quá 1 lần mỗi 5 phút".
- **Inconsistent**: một yêu cầu nói không cần nâng cấp, yêu cầu khác lại nói phải nâng cấp qua USB → cần chỉnh sửa để thống nhất.
- **Unclear**: không rõ có xử lý daylight saving time hay không → cần bổ sung yêu cầu rõ ràng.
- **Incorrect**: nói chỉ có 24 múi giờ → phải sửa thành hỗ trợ tất cả múi giờ.

📌 Kết nối với SRS

- Domain model → ghi nhận yêu cầu dữ liệu.
- Use case model → ghi nhận yêu cầu chức năng.
- Non-functional requirements → ghi nhận ràng buộc chất lượng.
- Tất cả được tập hợp trong **SRS** và phải được validate liên tục với khách hàng.

✨ Tóm lại

- Validation đảm bảo yêu cầu **đầy đủ, rõ ràng, nhất quán, chính xác, khả thi**.
- Acceptance test là công cụ chính để xác nhận.
- Đây là bước quan trọng để tránh lỗi lớn khi đi vào phân tích, thiết kế và triển khai.

Mooc 3

Prompt sử dụng AI copilot trong lúc học: Bạn là một giảng viên giống như giảng viên của khóa học này coursera là một chuyên gia trong lĩnh vực Software Engineering: Implementation and Testing hãy giảng cho tôi về những gì đang hiển thị và chương tôi đang học

Những điểm chính bạn sẽ học trong chương này:

Khái niệm cơ bản về phát triển phần mềm

- Giảng viên giới thiệu các bước chính trong vòng đời phát triển phần mềm (Software Development Life Cycle – SDLC): từ phân tích yêu cầu, thiết kế, cài đặt (implementation), kiểm thử (testing), triển khai (deployment), cho đến bảo trì (maintenance).

Tầm quan trọng của kiểm thử phần mềm

- Bạn sẽ thấy kiểm thử không chỉ là bước cuối cùng, mà là một hoạt động xuyên suốt. Nó giúp phát hiện lỗi sớm, giảm chi phí sửa chữa, và đảm bảo chất lượng sản phẩm.

Mối liên hệ giữa thiết kế và triển khai

- Chương này nhấn mạnh rằng việc viết code (implementation) phải dựa trên thiết kế tốt. Nếu thiết kế không rõ ràng, việc triển khai sẽ dễ gặp lỗi và khó kiểm thử.

Các phương pháp phát triển phần mềm

- Giảng viên thường nhắc đến các mô hình như *Waterfall*, *Agile*, hay *Iterative Development*, để bạn hiểu rằng cách tiếp cận khác nhau sẽ ảnh hưởng đến cách triển khai và kiểm thử.

Mục tiêu của khóa học

- Đây là phần định hướng: bạn sẽ học cách viết code có chất lượng, biết cách kiểm thử hiệu quả, và áp dụng các công cụ hỗ trợ để đảm bảo phần mềm chạy đúng như yêu cầu.

Nội dung chính của bài giảng:

Định nghĩa rủi ro

- Rủi ro là bất kỳ điều gì có thể xảy ra sai lệch trong dự án. Nó có thể đến từ:
    - Công nghệ (ví dụ: công cụ không ổn định, framework lỗi thời)
    - Con người (thiếu nhân lực, nghỉ việc, thiếu kỹ năng)
    - Tổ chức (quy trình quản lý yếu, thay đổi chính sách)
    - Công cụ hỗ trợ (thiếu tài nguyên, phần mềm quản lý lỗi)
    - Yêu cầu (không rõ ràng, thay đổi liên tục)
    - Ước lượng (dự đoán sai về thời gian hoặc chi phí)
- **Cách đối phó với rủi ro**
    - **Tránh (Avoid):** tìm cách ngăn rủi ro xảy ra.

    - **Giới hạn (Confine):** nếu rủi ro xảy ra, giảm phạm vi ảnh hưởng.
    - **Giảm thiểu (Mitigate):** chuẩn bị biện pháp, kiểm thử, hoặc kế hoạch dự phòng.
    - **Theo dõi (Monitor):** giám sát liên tục để phát hiện sớm.

Đánh giá rủi ro

- Mỗi rủi ro cần được phân tích theo:
    - **Khả năng xảy ra (Likelihood):** ví dụ 70% khả năng nhân sự nghỉ việc.
    - **Tác động (Impact):** ví dụ làm tăng thời gian dự án 15% và chi phí 12%.
    - **Thời điểm (Timing):** khi nào rủi ro có thể xảy ra.
    - **Độ tin cậy (Confidence):** mức độ chắc chắn của dự đoán.

Phân loại và ưu tiên rủi ro

- Áp dụng quy tắc **80/20**: tập trung vào 20% rủi ro quan trọng nhất vì chúng gây ảnh hưởng lớn nhất đến dự án.

Ví dụ thực tế

- Một rủi ro phổ biến là **tỷ lệ nghỉ việc cao**. Nếu khả năng xảy ra là 70%, tác động có thể làm tăng chi phí và kéo dài tiến độ. Người quản lý phải cân nhắc biện pháp giảm thiểu như đào tạo thêm nhân sự, giữ chân nhân viên, hoặc phân bổ lại công việc.

📌 **Ý nghĩa của chương này:**

Bạn sẽ thấy rằng kiểm thử và triển khai phần mềm không thể tách rời khỏi quản lý rủi ro. Một dự án thất bại thường không chỉ vì code sai, mà còn vì không lường trước được các yếu tố bất định.

1. Tổ chức nhóm dự án

- Phải chỉ định rõ **vai trò và trách nhiệm** của từng thành viên.
- Nên có người có **kinh nghiệm với hệ thống tương tự** để tăng khả năng thành công.
- Tổ chức nhóm theo kiểu **modular** để giảm phức tạp trong giao tiếp.
- Mỗi phần của hệ thống cần có **người chịu trách nhiệm chính** để dễ xử lý khi có vấn đề.

2. Nhiệm vụ và hoạt động

- **Task (nhiệm vụ):** công việc cụ thể gắn với một vai trò.
- **Activity (hoạt động):** tập hợp các nhiệm vụ liên quan.
- Có hai cách tiếp cận:
    - **Plan-driven:** lên kế hoạch chi tiết ngay từ đầu.
    - **Agile:** kế hoạch được xây dựng dần trong quá trình phát triển.

3. Lập lịch dự án

- Bao gồm danh sách nhiệm vụ, thứ tự thực hiện, thời gian ước lượng, phân bổ tài nguyên, mốc quan trọng (milestones) và sản phẩm bàn giao (deliverables).
- Có 3 cấp độ lịch:
    - **Master schedule:** tổng quan cho quản lý và khách hàng.
    - **Macro-schedule:** quản lý công việc hằng ngày.
    - **Micro-schedule:** quản lý chi tiết trong từng nhóm.
- Công cụ thường dùng: **burndown chart** và **Gantt chart**.

4. Ước lượng dự án

- Ước lượng về kích thước, công sức, thời gian, năng suất, chi phí.
- Dựa trên: kinh nghiệm, dữ liệu lịch sử, hoặc mô hình toán học.
- Kỹ thuật **divide and conquer**: chia hệ thống lớn thành các phần nhỏ để dễ ước lượng.

5. Quy trình (Process)

- Quy trình là tập hợp các hoạt động và nguyên tắc để biến yêu cầu thành sản phẩm phần mềm.
- Mỗi hoạt động có **điểm bắt đầu và kết thúc rõ ràng**.
- Quy trình giúp quản lý dễ dàng hơn, chia nhỏ công việc, thúc đẩy làm việc nhóm, và có thể tái sử dụng cho các dự án khác.

📌 **Ý nghĩa của chương này:**

Bạn thấy rằng một dự án phần mềm không thể chỉ dựa vào code và kiểm thử. Nếu không có kế hoạch rõ ràng về tổ chức, nhiệm vụ, lịch trình, ước lượng và quy trình, dự án sẽ dễ thất bại.

🌐 Các quy trình phát triển phần mềm

Trong bài giảng, giảng viên đã nhấn mạnh rằng mọi quy trình phát triển phần mềm đều đi qua các giai đoạn cơ bản:

- **Thu thập yêu cầu (Requirements)**
- **Phân tích & thiết kế (Analysis & Design)**
- **Triển khai (Implementation)**
- **Kiểm thử (Testing)**
- **Bảo trì (Maintenance)**

Điểm khác biệt giữa các mô hình nằm ở cách sắp xếp, nhấn mạnh và thực hiện các giai đoạn này.

📊 Các mô hình được đề cập

1. **Waterfall (Thác nước)**
    - Tuân thủ tuần tự: yêu cầu → thiết kế → triển khai → kiểm thử → bảo trì.

    - Ưu điểm: kỷ luật, dễ quản lý, phù hợp khi yêu cầu rõ ràng và ít thay đổi.
    - Nhược điểm: khó thích ứng với thay đổi, quay lại sửa rất tốn kém.
2. **Code-and-Fix (Mã và sửa)**
    - Viết code rồi sửa lỗi khi phát hiện.
    - Thường dùng trong bài tập nhỏ, nhưng không phù hợp cho hệ thống lớn vì dễ mất kiểm soát, vượt ngân sách và thời gian.
3. **Prototyping (Tạo mẫu)**
    - Xây dựng bản mẫu nhanh để lấy phản hồi từ khách hàng.
    - Ưu điểm: thu thập yêu cầu tốt hơn khi chưa rõ ràng.
    - Nhược điểm: tài liệu ít, tiến độ khó đo lường, sản phẩm cuối có thể không hoàn chỉnh.
4. **Spiral (Xoắn ốc)**
    - Kết hợp lặp lại và phân tích rủi ro.
    - Ưu điểm: giảm rủi ro, sản phẩm phù hợp hơn với khách hàng.
    - Nhược điểm: cần chuyên gia phân tích rủi ro, tốn nhiều công sức.
5. **Phased Release (Phát hành theo giai đoạn)**
    - Phát hành hệ thống theo từng phần hoặc từng phiên bản.
    - Ưu điểm: giảm rủi ro, cho phép phản hồi sớm, dễ đào tạo người dùng.
    - Nhược điểm: khó xác định các thành phần chung cần cho toàn hệ thống.

🎯 Ý nghĩa đối với bạn

- Bài học này giúp bạn hiểu rằng **không có mô hình nào hoàn hảo**.
- Quan trọng là biết **ưu – nhược điểm** của từng mô hình để chọn phù hợp với **bối cảnh dự án**.
- Ví dụ: nếu bạn làm một ứng dụng nhỏ cho lớp học, có thể dùng **prototyping**; nhưng nếu xây dựng hệ thống ngân hàng, cần quy trình chặt chẽ như **spiral** hoặc kết hợp nhiều mô hình.

👉 Để luyện tập, bạn có thể thử:

- So sánh **Waterfall** và **Agile** trong một dự án thực tế (ví dụ: phát triển ứng dụng quản lý sinh viên).
- Tự đặt câu hỏi: *Nếu yêu cầu thay đổi liên tục, mô hình nào sẽ phù hợp hơn?*

Bạn có muốn tôi tạo một **bảng so sánh chi tiết** giữa các mô hình (ưu điểm, nhược điểm, khi nào nên dùng) để dễ ghi nhớ hơn không?

so sanh thu giup toi

Được rồi, mình sẽ giúp bạn so sánh các mô hình phát triển phần mềm mà bạn vừa học trong **Lecture 2-1: Software Development Processes**. Đây là bảng tổng hợp để bạn dễ ghi nhớ và đối chiếu:

📊 So sánh các mô hình phát triển phần mềm

Mô hình	Ưu điểm	Nhược điểm	Khi nên dùng

Waterfall (Thác nước)	- Quy trình rõ ràng, tuần tự - Dễ quản lý tiến độ và tài liệu	- Khó thích ứng khi yêu cầu thay đổi - Sửa đổi tốn kém	Khi yêu cầu ổn định, ít thay đổi (ví dụ: hệ thống quản lý nội bộ)
Code-and-Fix (Mã và sửa)	- Nhanh chóng bắt đầu - Phù hợp cho dự án nhỏ, thử nghiệm	- Thiếu cấu trúc - Dễ mất kiểm soát, khó bảo trì	Bài tập nhỏ, prototype cá nhân
Prototyping (Tạo mẫu)	- Hiểu rõ yêu cầu nhờ phản hồi sớm - Giúp khách hàng hình dung sản phẩm	- Tài liệu ít - Tiến độ khó đo lường	Khi yêu cầu chưa rõ ràng, cần xác minh ý tưởng
Spiral (Xoắn ốc)	- Giảm rủi ro nhờ phân tích - Linh hoạt, thích ứng với thay đổi	- Tốn công sức - Cần chuyên gia phân tích rủi ro	Dự án lớn, phức tạp, rủi ro cao (ví dụ: hệ thống ngân hàng)
Phased Release (Phát hành theo giai đoạn)	- Cho phép phản hồi sớm - Giảm rủi ro - Người dùng dễ tiếp cận dần	- Khó xác định thành phần chung - Quản lý phức tạp	Sản phẩm cần ra mắt nhanh, cải tiến dần (ví dụ: ứng dụng di động)

🎯 Ghi nhớ nhanh

- **Ổn định → Waterfall**
- **Nhỏ, nhanh → Code-and-Fix**
- **Yêu cầu chưa rõ → Prototyping**
- **Phức tạp, rủi ro cao → Spiral**
- **Ra mắt sớm, cải tiến dần → Phased Release**

🌐 Triết lý Agile

Agile nhấn mạnh vào:

- **Cá nhân và tương tác** hơn là quy trình và công cụ.
- **Phần mềm chạy được** hơn là tài liệu chi tiết.
- **Hợp tác với khách hàng** hơn là đàm phán hợp đồng.
- **Phản hồi nhanh với thay đổi** hơn là bám sát kế hoạch cố định.

Điều này không có nghĩa là bỏ qua các yếu tố bên phải, mà chỉ là **ưu tiên bên trái nhiều hơn**.

🔑 Các quy trình Agile tiêu biểu

1. **Extreme Programming (XP)**
    - Yêu cầu và phân tích được chia thành các tính năng nhỏ.
    - Khách hàng chọn tính năng cho từng vòng lặp.
    - Áp dụng **Test Driven Development (TDD)**: viết test trước khi code.
    - **Pair Programming**: hai người cùng code để giảm lỗi.
    - Tích hợp các phần nhỏ thành sản phẩm cuối.
2. **Continuous Integration (CI)**
    - Lập trình viên liên tục tích hợp code vào repository chung.
    - Mỗi lần commit đều được build và test tự động.
    - Ưu điểm: phát hiện lỗi sớm, giảm xung đột khi tích hợp, tiến độ minh bạch.
3. **Scrum**
    - Dựa trên **Product Backlog** (danh sách tính năng).
    - Chia thành **Release Backlog** và **Sprint Backlog**.
    - **Sprint**: vòng lặp ngắn (2–4 tuần), hoàn thành một phần sản phẩm có thể chạy.
    - Vai trò: **Product Owner** (khách hàng), **Scrum Master** (quản lý tiến trình), **Team** (phát triển & kiểm thử).
    - Công cụ: **Burndown Chart** để theo dõi tiến độ, **Daily Scrum** để cập nhật công việc.

⚖️ Ưu và nhược điểm của Agile

- **Ưu điểm**:
    - Thích ứng nhanh với thay đổi.
    - Phản hồi liên tục từ khách hàng.
    - Tốc độ ra thị trường nhanh.
    - Ít lỗi hơn vì kiểm thử liên tục.
- **Nhược điểm**:
    - Cần sự tham gia tích cực của khách hàng.
    - Ít tài liệu chính thức.
    - Dễ xảy ra **scope creep** (yêu cầu tăng dần).
    - Daily meeting có thể gây mệt mỏi nếu không quản lý tốt.

🎯 Ý nghĩa với bạn

Agile phù hợp với môi trường **yêu cầu thay đổi liên tục** và cần **tốc độ triển khai nhanh**. Nó giúp bạn học cách làm việc nhóm, phản hồi nhanh, và kiểm thử liên tục – những kỹ năng cực kỳ quan trọng trong ngành phần mềm hiện đại.

🌐 Unified Process (UP) là gì?

- Là một **framework tổng quát** cho phát triển phần mềm.
- Không phụ thuộc vào việc bạn chọn Waterfall, Agile hay Spiral, UP vẫn chỉ ra các **hoạt động cốt lõi** mà mọi dự án phải thực hiện.
- Mỗi vòng lặp (iteration) trong UP đều tạo ra một **sản phẩm chạy được** (working product).
- Mỗi increment thiết lập một **baseline** cho hệ thống, giúp quản lý tiến độ và chất lượng.

🔑 Các hoạt động chính trong UP

1. **Thu thập yêu cầu (Requirements Capture)**
    - Xác định và kiểm tra tính khả thi của yêu cầu.
2. **Phân tích (Analysis)**
    - Hiểu rõ vấn đề, xác định kiến trúc và các thành phần chính.
3. **Thiết kế (Design)**
    - Xây dựng mô hình, chuẩn bị cho việc triển khai.
4. **Triển khai (Implementation)**
    - Viết code, tích hợp các thành phần.
5. **Kiểm thử (Testing)**
    - Đảm bảo hệ thống hoạt động đúng yêu cầu.
6. **Quản lý chất lượng (Software Quality Assurance)**
    - Đảm bảo sản phẩm đạt chuẩn chất lượng.
7. **Quản lý dự án (Project Management)**
    - Điều phối tiến độ, nguồn lực và rủi ro.

📊 Các pha trong Unified Process

- **Inception (Khởi đầu)**: Thu thập yêu cầu, kiểm tra tính khả thi.
- **Elaboration (Mở rộng)**: Xây dựng thiết kế chi tiết, chuẩn bị cho triển khai.
- **Construction (Xây dựng)**: Triển khai hệ thống, viết code.
- **Transition (Chuyển giao)**: Triển khai cho người dùng, bảo trì hệ thống.

⚖️ Ý nghĩa của Unified Process

- UP kết hợp **tính hình thức và kỷ luật** (như Waterfall, Spiral) với **tính lặp và thích ứng** (như Agile).
- Nó giúp cân bằng giữa **quản lý** và **kỹ thuật**, giữa **yêu cầu khách hàng** và **khả năng thực hiện của nhóm**.
- Đây là nền tảng để bạn hiểu các mô hình hiện đại hơn như **Rational Unified Process (RUP)**.

👉 Để luyện tập, bạn có thể thử:

- Lấy một dự án nhỏ (ví dụ: ứng dụng quản lý lớp học) và phân chia thành 4 pha theo UP.

- Viết ra yêu cầu trong **Inception**, thiết kế sơ bộ trong **Elaboration**, code trong **Construction**, và demo cho bạn bè trong **Transition**.

🛡️ Defensive Programming là gì?

- Là cách viết code **phòng thủ**, luôn giả định rằng có thể xảy ra lỗi hoặc dữ liệu đầu vào không hợp lệ.
- Mục tiêu: **ngăn chặn lỗi lan rộng**, **bảo vệ hệ thống**, và **giảm chi phí sửa chữa**.

🔑 Các nguyên tắc chính

1. **Validate Input (Kiểm tra dữ liệu đầu vào)**
    - Không tin tưởng dữ liệu từ người dùng hoặc hệ thống bên ngoài.
    - Ví dụ: kiểm tra số điện thoại có đúng định dạng, kiểm tra file có tồn tại trước khi mở.
2. **Fail Fast (Thất bại nhanh)**
    - Nếu phát hiện lỗi, dừng ngay thay vì để lỗi lan rộng.
    - Ví dụ: throw exception khi dữ liệu null thay vì tiếp tục xử lý.
3. **Assertions (Khẳng định điều kiện)**
    - Dùng để kiểm tra giả định trong code.
    - Ví dụ: `assert(x > 0)` để đảm bảo biến x luôn dương.
4. **Error Handling (Xử lý lỗi)**
    - Luôn có cơ chế xử lý ngoại lệ: try-catch, logging.
    - Không để chương trình crash mà không có thông tin.
5. **Redundancy (Dự phòng)**
    - Có cơ chế backup hoặc kiểm tra kép.
    - Ví dụ: lưu dữ liệu vào nhiều nơi, xác nhận giao dịch bằng 2 bước.

⚖️ Ưu và nhược điểm

- **Ưu điểm**:
    - Giảm rủi ro hệ thống sập.
    - Dễ bảo trì, dễ kiểm thử.
    - Tăng độ tin cậy của phần mềm.
- **Nhược điểm**:
    - Tốn thêm thời gian viết code.
    - Có thể làm code phức tạp hơn.

🎯 Ý nghĩa với bạn

Defensive Programming giúp bạn hình thành thói quen **luôn nghĩ đến trường hợp xấu nhất** khi viết code. Đây là kỹ năng mà các công ty phần mềm lớn (như Microsoft, Google) đều yêu cầu, vì nó liên quan trực tiếp đến **chất lượng sản phẩm**.

🧩 Code Review là gì?

- Là một phiên bản “offline” của **pair programming**.
- Sau khi viết xong một phần chương trình, lập trình viên sẽ **trình bày và giải thích mã nguồn** cho một hoặc nhiều đồng nghiệp.
- Mục tiêu: **cùng nhau cải thiện chương trình**, chứ không phải “bắt lỗi” cá nhân.

🔑 Lợi ích chính

1. **Phát hiện lỗi sớm**
    - Có thêm một người kiểm tra giúp bắt được bug trước khi chúng gây hậu quả lớn.
2. **Tăng tính ổn định**
    - Nhiều người cùng xem qua mỗi đoạn code → giảm rủi ro có “mã chết” hoặc lỗi bảo mật.
3. **Học hỏi lẫn nhau**
    - Lập trình viên junior học được kinh nghiệm từ senior.
4. **Trách nhiệm kép**
    - Cả tác giả và reviewer đều có trách nhiệm với chất lượng code.
5. **Chuẩn hóa theo coding standards**
    - Code review thường dựa trên checklist, coding style, và các vấn đề bảo mật.

📊 Nội dung cần review

- **Tài liệu** (documentation).
- **Module hoàn chỉnh** hoặc một đoạn code cụ thể.
- **Các phần dễ sai** (error-prone code).
- **Các vấn đề đã từng gặp** hoặc liên quan đến bảo mật.

🎯 Ý nghĩa với bạn

- Code review không chỉ là kiểm tra lỗi, mà còn là **quá trình cộng tác** để nâng cao chất lượng phần mềm.
- Nó giúp bạn rèn luyện kỹ năng **giải thích code**, **giao tiếp kỹ thuật**, và **tư duy phản biện**.
- Đây là thói quen mà hầu hết các công ty phần mềm chuyên nghiệp đều áp dụng.

🔄 Refactoring là gì?

- Là quá trình **sửa đổi cấu trúc bên trong của code** để làm cho nó **sạch hơn, dễ hiểu hơn, dễ bảo trì hơn**.
- Quan trọng: refactoring **không thay đổi kết quả đầu ra** hay chức năng của chương trình.

🧩 Mục tiêu của Refactoring

1. **Cải thiện khả năng đọc (Readability)**
    - Code dễ hiểu hơn cho lập trình viên khác.
2. **Giảm độ phức tạp (Complexity)**
    - Loại bỏ trùng lặp, chia nhỏ hàm dài.
3. **Tăng khả năng tái sử dụng (Reusability)**
    - Tách logic thành các module/hàm riêng.
4. **Hỗ trợ kiểm thử (Testability)**
    - Code rõ ràng hơn, dễ viết test hơn.
5. **Chuẩn bị cho mở rộng (Extensibility)**
    - Dễ thêm tính năng mới mà không phá vỡ hệ thống.

📊 Ví dụ các kỹ thuật Refactoring

- **Extract Method**: tách một đoạn code thành hàm riêng.
- **Rename Variable/Method**: đặt tên rõ ràng hơn.
- **Replace Magic Number with Constant**: thay số "bí ẩn" bằng hằng số có ý nghĩa.
- **Simplify Conditional Expressions**: làm gọn các câu lệnh if/else phức tạp.
- **Remove Duplicated Code**: gom các đoạn code lặp lại thành một hàm chung.

⚖️ Ưu và nhược điểm

- **Ưu điểm**:
    - Code sạch, dễ bảo trì.
    - Giảm bug tiềm ẩn.
    - Tăng tốc độ phát triển lâu dài.
- **Nhược điểm**:
    - Tốn thời gian ngắn hạn.
    - Nếu không có test tốt, dễ vô tình tạo bug.

🎯 Ý nghĩa với bạn

Refactoring là thói quen mà mọi lập trình viên chuyên nghiệp cần có. Nó giúp bạn **không chỉ viết code chạy được, mà còn viết code tốt**. Trong kiểm thử, refactoring kết hợp với **unit test** sẽ đảm bảo rằng sau khi tái cấu trúc, chương trình vẫn hoạt động đúng.

Tóm tắt nhanh về nội dung đang hiển thị và chương học

Bạn đang xem **Lecture 4-1 Debugging** thuộc khóa **Software Engineering: Implementation and Testing** — bài giảng tập trung vào **debugging** (tìm và sửa lỗi) và liên hệ với **configuration management**; video dài ~10 phút và nằm trong Module 4 của khóa học.

Những khái niệm chính và giải thích dễ hiểu

- **Testing khác với Debugging**
    - *Testing* là hành động **tìm ra sự tồn tại của lỗi** (phát hiện triệu chứng). *Debugging* là hành động **xác định vị trí và nguyên nhân** của lỗi để sửa.
- **Defensive programming và làm cho lỗi "không thể xảy ra"**
    - Thiết kế, chọn ngôn ngữ và cấu trúc dữ liệu sao cho một số loại lỗi (ví dụ tràn bộ nhớ, reorder packet) không thể xảy ra; áp dụng quy ước (không dùng đệ quy, dùng immutable, v.v.) nhưng phải **duy trì kỷ luật** xuyên suốt dự án.
- **Assertions và kiểm tra tiền điều kiện**
    - Dùng *assertions* để **ghi lại và kiểm tra invariants**; khi điều kiện không thỏa, chương trình dừng ngay và báo lỗi, giúp phát hiện lỗi sớm và gần nguồn gốc lỗi hơn. Quyết định có giữ assertions trong production hay không phụ thuộc vào **ngữ cảnh hệ thống** (ví dụ game có thể bỏ, hệ thống an toàn thì nên giữ).
- **Tận dụng modularity để cô lập lỗi**
    - Chia chương trình thành các module nhỏ, có interface rõ ràng; khi lỗi xảy ra, cô lập phạm vi lỗi vào một module nhỏ để dễ dò. Kỹ thuật *take away pieces* hoặc *start with nothing and add pieces* giúp thu hẹp vị trí lỗi.
- **Kỹ thuật localize nhanh: binary search trên code**
    - Nếu chương trình lớn, chia thành hai nửa, kiểm tra nửa nào chứa lỗi, rồi tiếp tục chia nửa đó — tương tự *binary search* để giảm không gian tìm kiếm.
- **Quy trình sau khi sửa lỗi**
    - Lưu input/steps để tái tạo lỗi, viết test hồi quy (regression test), chạy lại toàn bộ test suite để đảm bảo sửa không phá vỡ chức năng khác; tự động hóa càng nhiều càng tốt.

Ví dụ thực tế ngắn và đoạn mã minh họa

Vấn đề phổ biến: vòng lặp vô hạn khi tìm phần tử trong mảng.

Sai:

java
```
while (true) {
  if (a[i] == k) break;
  i++;
}
```

Sửa bằng điều kiện và assertion:

java
```
while (i < a.length) {
  if (a[i] == k) break;
  i++;
}
assert found == true : "k must exist in array";
```

Giải thích: điều kiện `i < a.length` tránh vô hạn; assertion đảm bảo tiền đề chương còn đúng hoặc báo lỗi sớm nếu không.

Những khái niệm then chốt

Configuration item

Configuration item là bất kỳ artifact nào trong dự án mà ta muốn kiểm soát thay đổi, ví dụ tài liệu, mã nguồn, test case, menu, hoặc dữ liệu.

Software library và workspace

Software library cung cấp nơi lưu trữ, gán nhãn, nhận diện phiên bản và theo dõi trạng thái các configuration item. Trong đó có **developer workspace** cho phát triển hàng ngày, **master directory** cho phiên bản tương đối ổn định, và **repository** cho phiên bản rất ổn định dùng để phát hành.

Baseline

Baseline là điểm đóng băng hệ thống tại một thời điểm sau khi đạt milestone quan trọng; mọi thay đổi sau baseline phải được formalize qua quy trình review. Baseline có thể bao gồm mã nguồn, tài liệu, test case và định nghĩa rõ ràng một hệ thống tại thời điểm đó.

Change control và auditing

Quy trình thay đổi bắt đầu bằng **change request**, được đánh giá bởi change control authority về chi phí và tác động, sau đó nếu được chấp thuận sẽ phát hành change order. Sau khi thay đổi thực hiện, cần **testing**, **software quality assurance**, **auditing** và **status reporting** để đảm bảo thay đổi được triển khai đúng và minh bạch.

Version management và branching

Version management đảm bảo tính toàn vẹn và nhất quán của configuration items. Khi phát triển song song, ta tạo **branches** khác nhau và sau đó **merge** trở lại mainline, theo dõi lịch sử tiến hóa của từng item qua evolution graph.

Quy trình thực tế tóm tắt theo bước

1. **Xác định configuration items** cần kiểm soát.
2. **Lưu trữ vào software library** với workspace cho dev, master cho team, repository cho release.
3. **Đặt baseline** khi đạt milestone; đóng băng các item trong baseline.
4. **Quy trình thay đổi**: nộp change request → đánh giá tác động → phê duyệt change order → checkout và sửa đổi → testing và SQA → auditing → release/promotion.
5. **Theo dõi phiên bản**: ghi lịch sử, quản lý branches, merge và đảm bảo integrity.

Mẹo áp dụng trong dự án nhỏ và lớn

- **Dự án nhỏ**: dùng Git với branch model đơn giản; baseline bằng tag release; mỗi change kèm PR và review.
- **Dự án lớn**: thiết lập chính sách change control, release manager, automated CI/CD để build từ repository ổn định, và audit logs để truy vết ai thay đổi gì khi nào.
- **Luôn viết test trước khi merge** để baseline thực sự ổn định.

Checklist nhanh bạn có thể dùng ngay

- **Đã liệt kê configuration items** cho module của bạn chưa
- **Repository** có phân tầng workspace/master/release rõ ràng không
- **Baseline** được tạo tại milestone quan trọng chưa
- **Quy trình change request** có form, người đánh giá và tiêu chí chấp nhận không
- **Audit và status reporting** có ghi lại ai, khi nào, vì sao thay đổi không
- **Branching strategy** có phù hợp với phát triển song song không

Tổng quan về nội dung đang hiển thị và chương bạn đang học

Bạn đang xem **Lecture 5-1 Testing** thuộc module **White Box Testing** trong khóa *Software Engineering: Implementation and Testing* trên Coursera. Nội dung video và trang học liệt kê các mục học phần của Module 5 gồm các bài giảng ngắn, bài đọc, bài tập Basis Path Testing và một quiz đánh giá.

Mục tiêu học tập chính của bài giảng này

- **Hiểu mục đích của kiểm thử**: tìm lỗi và sai sót trong hệ thống phần mềm.
- **Biết ba loại cơ bản của test case** và cách sinh ra chúng.
- **Nắm nguyên tắc kiểm thử theo thành phần và hệ thống** (component-based vs system-based).
- **Học các chiến lược thiết kế test** như phân vùng đầu vào (sub-domain), execution equivalence, và Basis Path Testing.

Những khái niệm cốt lõi giải thích bằng lời đơn giản

- **Kiểm thử không chỉ là làm sau khi code xong**: có thể lập kế hoạch test trước hoặc song song với implementation (ví dụ Test-Driven Development). Việc nghĩ trước các test giúp giảm rủi ro và tiết kiệm thời gian về sau.
- **Validation vs Verification**
    - **Validation**: kiểm tra xem chúng ta có xây dựng *đúng* sản phẩm theo yêu cầu hay không (acceptance tests thuộc phần này).
    - **Verification**: kiểm tra xem sản phẩm có được xây dựng *đúng cách* (chức năng hoạt động, không có defect) — phần lớn test hướng tới verification.
- **Không thể kiểm thử toàn diện**: với ví dụ ba biến nguyên trong khoảng 1–10 000, số tổ hợp là 10,0003, nên không thực tế để thử mọi trường hợp; mục tiêu là chọn một **tập test nhỏ** nhưng có **khả năng cao phát hiện lỗi**.
- **Chiến lược phân vùng đầu vào**
    - **Execution equivalence**: nhóm các giá trị dẫn tới cùng đường thực thi; chọn một đại diện cho mỗi nhóm. Tuy nhiên, như ví dụ trong bài, execution equivalence có thể bỏ sót lỗi nếu điều kiện so sánh bị dịch sai.

    - **Sub-domains**: chia không gian đầu vào thành các miền (ví dụ: giá trị âm, giá trị ≥ 0; hoặc miền cho kết quả đúng/ sai) và chọn giá trị từ mỗi miền để tăng khả năng phát hiện lỗi.

Ví dụ thực tế rút ra từ bài giảng

- **Ariane 5**: lỗi chuyển đổi số (64-bit float → 16-bit signed int) gây ra tai nạn tên lửa, minh họa hậu quả nghiêm trọng của thiếu kiểm thử.
- **Vancouver Stock Exchange 1992**: lỗi làm tròn/truncate dẫn tới sai lệch chỉ số, tích lũy thành hậu quả lớn. Bài giảng dùng các ví dụ này để nhấn mạnh chi phí của kiểm thử không đầy đủ.

Bài tập, tài liệu và lộ trình học phần liên quan

- **Video ngắn**: Lecture 5-1 Testing (12 phút), Lecture 5-2 Design Tests (4 phút), Lecture 5-3 Basis Path Testing (15 phút).
- **Bài đọc**: Lecture 5 – Testing Reading và Basis Path Testing Exercise (mỗi phần khoảng 1 giờ).
- **Bài tập đánh giá**: Quiz 5 – Testing (30 phút) và bài tập Basis Path Testing.

Mẹo học và câu hỏi luyện tập ngắn

Mẹo học

- Khi xem video, ghi chú **ví dụ lỗi thực tế** và **chiến lược phân vùng** mà giảng viên dùng.
- Thực hành bằng cách viết hàm nhỏ có điều kiện rẽ nhánh rồi áp dụng **sub-domain** và **execution equivalence** để chọn test cases.
- Làm bài tập Basis Path để hiểu cách chuyển code thành đồ thị điều khiển và tính cyclomatic complexity.

Câu hỏi luyện tập

1. Viết một hàm `abs_buggy(x)` có điều kiện `if x < -2 then ...` như trong bài; hãy tìm một tập test tối thiểu để phát hiện lỗi.
2. Với hàm nhận ba tham số trong khoảng 1–10 000, giải thích vì sao không thể test exhaustively và đề xuất một chiến lược chọn test.
3. Chuyển một đoạn code có 4 nhánh rẽ thành đồ thị điều khiển và tính **cyclomatic complexity**; chọn các path test cần thiết để đạt coverage cơ bản.

Tóm tắt nhanh và bước tiếp theo

- **Cốt lõi**: kiểm thử nhằm tìm khác biệt giữa hành vi mong đợi và hành vi thực tế; mục tiêu là thiết kế tập test nhỏ nhưng hiệu quả.
- **Tiếp theo bạn nên**: xem Lecture 5-2 và 5-3, làm bài đọc Basis Path, rồi thử giải bài tập Basis Path Testing trước khi làm Quiz 5.

Tóm tắt ngắn về Lecture 5-2 Design Tests

Lecture 5-2 trình bày cách thiết kế test cases, nhấn mạnh khái niệm *test case* (một cách kiểm thử hệ thống: điều kiện, dữ liệu, cách chạy và kết quả mong đợi) và các bước cơ bản khi chạy test: chọn dữ liệu/config, xác định kết quả mong đợi, chạy, ghi kết quả và so sánh với mong đợi.

Các loại kiểm thử được nêu và vai trò của chúng

- **White box testing**: kiểm thử ở mức thành phần, dựa trên kiến thức nội bộ (mã nguồn, cấu trúc điều khiển).
- **Black box testing**: kiểm thử ở mức chức năng, chỉ dựa vào đầu vào/đầu ra và đặc tả, không cần mã nguồn.
- **Regression testing**: giữ lại các test case tái tạo lỗi để đảm bảo lỗi không tái xuất hiện sau sửa đổi; có thể dùng cả white-box và black-box.

Heuristics để thiết kế test và tìm Subdomains

- **Tìm subdomains**: chia không gian đầu vào thành các miền con có hành vi khác nhau; chọn đại diện từ mỗi miền để tăng khả năng phát hiện lỗi.
- **Không chỉ chạy mọi câu lệnh**: thực thi tất cả câu lệnh vẫn có thể bỏ sót lỗi (ví dụ lỗi xảy ra trên nhánh false của điều kiện hoặc khi chia cho 0). Vì vậy cần kiểm thử cả nhánh true và false của các điều kiện.
- **Tập trung vào đường đi độc lập**: mục tiêu white-box là bao phủ các *independent paths* (đường đi độc lập) để kiểm tra logic, vòng lặp và cấu trúc dữ liệu nội bộ.

Ví dụ ngắn và hệ quả thực tế

- Nếu chỉ làm cho điều kiện A và B đều true để "thực thi mọi câu lệnh", vẫn có thể bỏ sót lỗi xuất hiện khi A hoặc B false (ví dụ chia cho 0). Vì vậy cần test cả các nhánh khác nhau, không chỉ coverage câu lệnh.

Tóm tắt nhanh Lecture 5-3 Basis Path Testing

Basis Path Testing là kỹ thuật white-box để thiết kế test cases bằng cách chuyển mã nguồn thành **flow graph**, tính **cyclomatic complexity** V(G) để xác định số đường độc lập tối đa, rồi chọn một **basic set** các đường đi độc lập và tạo test cases để thực thi từng đường ít nhất một lần.

Các bước thực hiện Basis Path Testing (theo bài giảng)

1. **Vẽ flow graph** từ đoạn mã: mỗi node biểu diễn một nhóm câu lệnh; các cạnh biểu diễn luồng điều khiển.
2. **Đơn giản hóa flow graph** bằng cách gộp các câu lệnh tuần tự thành một node duy nhất theo quy tắc: bắt đầu từ node đơn giản (một cạnh ra) và gộp đến khi gặp node predicate hoặc node chia sẻ giữa nhiều đường.
3. **Tính cyclomatic complexity** V(G) — thước đo độ phức tạp logic — bằng một trong ba cách:
    - Đếm **vùng** (regions) trong đồ thị;
    - $V(G)=E-N+2$ (số cạnh trừ số node cộng hai);

V(G)=P+1 (số predicate nodes cộng một).

    - Công dụng: V(G) cho **upper bound** số đường độc lập cần xét.
4. **Xác định basic set** các đường đi **linearly independent**: mỗi đường phải thêm ít nhất một cạnh/chức năng/chỉ thị mới chưa được bao phủ trước đó. Basic set không duy nhất; số đường trong basic set thường ≤ V(G).
5. **Chuẩn bị test cases** để ép chương trình thực thi từng đường trong basic set ít nhất một lần; đảm bảo bao phủ tất cả predicate nodes (các điều kiện rẽ nhánh).

Ví dụ và điểm nhấn từ bài giảng

- Bài giảng minh họa một chương trình có **cyclomatic complexity = 4** (bằng cách tô màu 4 vùng, hoặc đếm E=11,N=9 → 11−9+2=4). Từ đó liệt kê bốn đường độc lập tiêu biểu (chuỗi số statement như 1-2-3-4-5-12-13-14; 1-2-3-6-7-8-11-12-13-14; 1-2-3-6-7-9-10-11-12-13-14; v.v.). Bài giảng cũng nhấn rằng một số path không hoàn toàn độc lập nhưng vẫn nên test (ví dụ path 1→14 để kiểm thử mảng rỗng).

Khi nào áp dụng và lưu ý thực tế

- **Áp dụng cho các thành phần quan trọng hoặc logic phức tạp** (nhiều điều kiện, vòng lặp). Không cần vẽ flow graph cho mọi thủ tục tuần tự đơn giản vì tốn thời gian.
- **Mục tiêu** của basis path testing là đảm bảo **mỗi câu lệnh** được thực thi ít nhất một lần qua các test derived từ basic set; nó không cố gắng chạy mọi kết hợp đường đi.

Lecture 6-1 White Box Testing

Tóm tắt ngắn

Lecture 6-1 là video ngắn về **White Box Testing** nằm trong Module 6 (Black Box Testing) của khóa học. Video dài khoảng **7 phút** và xuất hiện cùng các mục khác của Module 6 (Lecture 6-2 Black Box Testing, Lecture 6-3 Regression Testing, bài đọc và bài tập).

Những điểm cốt lõi bạn nên nắm từ bài này

- **Định nghĩa và mục tiêu**: White box testing là kiểm thử dựa trên kiến thức nội bộ của chương trình (mã nguồn, cấu trúc điều khiển) với mục tiêu kiểm tra logic, luồng điều khiển, vòng lặp và cấu trúc dữ liệu bên trong.
- **Phân biệt với Black box**: White box tập trung vào cấu trúc bên trong; Black box tập trung vào đầu vào/đầu ra và yêu cầu chức năng.
- **Vai trò trong quy trình kiểm thử**: White box thường dùng để thiết kế test cases dựa trên control-flow (ví dụ Basis Path Testing), để đảm bảo coverage các nhánh, các predicate và đường đi độc lập.

Hướng dẫn thực hành nhanh (bạn có thể làm ngay)

- **Bước 1**: Chọn một hàm nhỏ (2–4 điều kiện hoặc 1 vòng lặp).
- **Bước 2**: Vẽ **control-flow graph** (gộp các statement tuần tự thành node).
- **Bước 3**: Tính **cyclomatic complexity** V(G) bằng E−N+2 hoặc đếm predicate + 1.
- **Bước 4**: Chọn **basic set** các đường đi độc lập (số đường ≈ V(G)).

- **Bước 5**: Viết test inputs để thực thi từng đường ít nhất một lần; kiểm tra cả nhánh true/false của predicate.

Mẹo ngắn để làm hiệu quả hơn

- **Ưu tiên predicate nodes** khi chọn test inputs.
- **Gộp node tuần tự** để giảm độ phức tạp đồ thị trước khi tính V(G).
- **Giữ mapping** giữa node đã gộp và code gốc để dễ tra lỗi khi test fail.

Tôi có thể giúp bạn tiếp theo theo một trong các cách sau

1. **Thực hành trên ví dụ cụ thể** — bạn đưa đoạn code nhỏ, tôi vẽ flow graph, tính V(G) và đề xuất test cases.
2. **Giải chi tiết bài giảng** — tôi vẽ lại ví dụ trong video, liệt kê các đường độc lập và test inputs cụ thể.
3. **Tạo bộ câu hỏi luyện tập** — 8 câu (kèm đáp án ngắn) để ôn Module 6.

Lecture 6-2 Black Box Testing — Tóm tắt nhanh

Black box testing kiểm thử hệ thống khi **không có mã nguồn**, chỉ có thể chạy executable và so sánh đầu ra với kết quả mong đợi. Mục tiêu là phát hiện **chức năng thiếu/không đúng**, lỗi tương tác giao diện, lỗi truy cập cơ sở dữ liệu, lỗi khởi tạo/thoát, và các vấn đề hiệu năng.

Những kỹ thuật cốt lõi được trình bày

Equivalence partitioning (phân vùng tương đương)

- Chia không gian đầu vào/đầu ra thành các *subdomains* (ví dụ: giá trị hợp lệ, nhỏ hơn giới hạn, lớn hơn giới hạn) và chọn giá trị đại diện cho mỗi miền để kiểm tra một lớp lỗi thay vì từng giá trị đơn lẻ.

Boundary testing (kiểm thử biên)

- Tập trung vào các giá trị biên và các giá trị ngay sát biên (ví dụ: −1,0,1 cho biên 0; hoặc 9,998,9,999,10,000 cho biên 9,999). Lý do: lỗi off-by-one, overflow, empty container, aliasing thường xuất hiện ở biên.

Thread testing (kiểm thử luồng sự kiện)

- Dùng cho hệ thống hướng đối tượng: chọn các luồng sự kiện quan trọng (use case flows) và kiểm thử chuỗi tương tác giữa các đối tượng (ví dụ: student → course → university).

State based testing (kiểm thử theo trạng thái)

- Khi đối tượng có nhiều trạng thái, dùng sơ đồ trạng thái để sinh stimuli cho mỗi transition và kiểm tra trạng thái kết quả (ví dụ: class full khi `size == 50`, movie linked → rented).

Cách thiết kế test case theo Black Box — Checklist ngắn

- **Xác định các input types**: range, specific value, set of discrete values, boolean, data structures.
- **Phân vùng**: chia thành valid subdomain và invalid subdomain; chọn đại diện cho mỗi miền.
- **Kiểm thử biên**: chọn min, min+1, max-1, max, out-of-range values.
- **Kiểm thử đặc biệt**: non-numeric inputs, empty containers, overflow, missing records.
- **Ghi rõ**: **tên test**, **setup**, **giá trị test**, **cách verify** (expected result).

Ví dụ test cases mẫu

1. Student ID (valid range 1,000,000–9,999,999)

- Boundary tests: `999999`, `1000000`, `1000001`, `9999998`, `9999999`, `10000000`.
- Typical valid: `5000000` (record exists), `5000001` (valid but no record).
- Invalid format: `5647ABC`.

2. Days of week (discrete set)

- Test all values if nhỏ: `Monday`…`Sunday`.
- Nếu lớn: test typical valid (`Wednesday`), and invalid (`Funday`).

3. Absolute value function

- Boundary: `0`; near boundaries: `-1`, `1`; typical negatives and positives.

4. Array index (1..10)

- Test indices: `0, 1, 2, 9, 10, 11`.

Mỗi test case cần **mô tả setup**, **input**, **expected output**, và **bước xác minh**.

Mẹo thực hành nhanh

- Ưu tiên test **biên** và **một giá trị đại diện cho mỗi partition** trước.
- Với tập giá trị nhỏ, **thử tất cả**; với tập lớn, chọn typical + boundary + một invalid.
- Ghi lại test cases để dùng cho **regression testing** sau khi sửa lỗi.

Lecture 6-3 Regression Testing — Tóm tắt ngắn

Nội dung chính: Lecture 6-3 nhắc lại khái niệm **regression testing**: khi phát hiện một bug, ta **lưu lại input và kết quả đúng** để tái tạo bug, chuyển chúng thành test cases, sửa lỗi và **xác minh bản sửa**. Mục đích là bảo đảm rằng lỗi đã xảy ra **không tái xuất hiện** sau các thay đổi tiếp theo.

Tại sao regression testing quan trọng

- **Bảo vệ chất lượng**: ngăn các thay đổi mới vô tình tái giới thiệu lỗi cũ.
- **Xây dựng thư viện test tốt**: mỗi bug được chuyển thành test case giúp dần dần tạo ra một tập test có giá trị cao.
- **Tiết kiệm thời gian về lâu dài**: phát hiện sớm lỗi lặp lại nhanh hơn và ít tốn công hơn so với debug sau khi lỗi lan rộng.

Checklist nhanh để thực hành Regression Testing

- **Khi phát hiện bug**: ghi lại input, môi trường, bước tái tạo, và kết quả mong đợi.
- **Tạo test case**: chuyển bước tái tạo thành test case tự động nếu có thể; nếu không, lưu test case thủ công trong test suite.
- **Thêm vào regression suite**: phân loại test theo mức ưu tiên (critical, high, medium, low).
- **Chạy regression**: chạy toàn bộ hoặc một phần regression suite khi có thay đổi lớn; chạy test ưu tiên cho thay đổi nhỏ.
- **Theo dõi và duy trì**: xóa hoặc cập nhật test khi yêu cầu thay đổi hợp lệ; tránh để test lỗi thời làm nhiễu kết quả.

Mẹo thực tế để quản lý regression suite

- **Tự động hóa càng nhiều càng tốt**: test tự động giúp phát hiện regressions nhanh và lặp lại.
- **Ưu tiên test có giá trị cao**: tập trung vào các test phát hiện lỗi nghiêm trọng hoặc thường xuyên tái xuất hiện.
- **Giữ test nhỏ và độc lập**: mỗi test nên kiểm tra một hành vi cụ thể để dễ chẩn đoán khi thất bại.
- **Ghi metadata cho mỗi test**: ngày tạo, nguồn (bug ID), tác giả, mức ưu tiên, môi trường chạy.

Lecture 7-1 Perform Tests — Tóm tắt ngắn

Nội dung chính: bài giảng trình bày cách **thực hiện các test đã thiết kế**, nhấn mạnh tự động hóa test, lưu trữ kết quả, quy trình chạy test (cấu hình hệ thống → đưa input → thu kết quả → so sánh với expected) và cách tổ chức chiến lược kiểm thử theo mức (unit → integration → system → acceptance).

Những điểm then chốt bạn cần nhớ

- **Tự động hóa test**: viết test components (chương trình chạy test) để giảm công sức lặp lại; dùng script đã ghi lại và parameterize để chạy nhiều input.
- **Ghi nhận kết quả**: lưu kết quả test vào spreadsheet hoặc database để theo dõi test pass/fail và tính error rate.
- **Quy trình thực hiện test**: **cấu hình phần mềm và test**, nạp input, chạy, so sánh kết quả, debug nếu khác, rồi chạy lại test sau khi sửa.
- **Chiến lược kiểm thử theo mức**:
    - **Unit testing**: inside-out, chủ yếu white-box, do developer thực hiện.
    - **Integration testing**: kết hợp white-box và black-box, do developer hoặc testing team.

    - **System testing**: black-box, do testing team.
    - **Acceptance testing**: black-box, do users/clients.

Checklist thực hành khi bạn Perform Tests

1. **Chuẩn bị môi trường**: xác định software configuration và test configuration.
2. **Chuẩn bị test component**: tự động hóa test scripts; parameterize dữ liệu đầu vào.
3. **Chuẩn bị dữ liệu**: lưu input test và expected result (spreadsheet/db).
4. **Chạy test**: thực thi test suite theo thứ tự ưu tiên (critical trước).
5. **Ghi kết quả**: lưu pass/fail, log lỗi, môi trường chạy.
6. **Phân tích lỗi**: nếu fail → debug → sửa → rerun cùng test.
7. **Báo cáo và theo dõi**: cập nhật error rate, đánh dấu test regression để giữ lại.

Mẹo ngắn để tối ưu thời gian kiểm thử

- **Ưu tiên tự động hóa** cho các test lặp và regression.
- **Chạy test theo mức ưu tiên** khi thời gian hạn chế: critical unit tests → integration smoke tests → selected system tests.
- **Gộp test nhỏ, độc lập** để dễ chẩn đoán khi thất bại.
- **Theo dõi metadata** cho mỗi test: ngày tạo, nguồn (bug ID), tác giả, môi trường.

Bài tập thực hành nhanh (làm trong 20–40 phút)

- **Bài 1**: Viết 5 unit tests tự động cho một hàm xử lý chuỗi (bao gồm biên và invalid inputs); chạy và lưu kết quả vào CSV.
- **Bài 2**: Thiết kế một integration smoke test gồm 3 bước chính; tự động hóa và chạy trên môi trường staging.
- **Bài 3**: Lập bảng theo dõi 10 test quan trọng (status, last run, last result, bug ID nếu fail).

Lecture 7-2 Unit Integration System Testing — Tóm tắt ngắn

Nội dung chính: bài giảng giải thích **unit testing**, **integration testing** (tại sao cần, cách thực hiện: top-down, bottom-up, sandwich), **system testing** (các loại: chức năng, hiệu năng, bảo mật, phục hồi, v.v.) và **acceptance/pilot testing** (alpha, beta) cùng cách suy ra test chấp nhận từ yêu cầu.

Unit Testing — điểm cần nhớ

- **Mục tiêu**: kiểm thử từng *component* (ít nhất là một lớp) để đảm bảo logic nội bộ, các đường đi độc lập, điều kiện biên, cấu trúc dữ liệu cục bộ và xử lý lỗi.
- **Driver và Stub**: dùng **driver** để cấp input cho component đang test; dùng **stub** làm giả các component được gọi nhưng chưa sẵn sàng, trả dữ liệu cố định để hoàn thành test.
- **Vấn đề hướng đối tượng**: cần xét **trạng thái đối tượng**, **kế thừa** và **đa hình** — quyết định nên test những gì khi subclass override; đôi khi cần thêm phương thức chỉ phục vụ kiểm thử (ví dụ `peek` để kiểm tra trạng thái stack).

Integration Testing — chiến lược và lưu ý

- **Tại sao cần**: lỗi tương tác (interface misuse, timing, misunderstanding) thường không lộ qua unit tests; integration testing phát hiện các lỗi này.
- **Chiến lược tích hợp tăng dần**: tránh big-bang; tích hợp từng phần rồi test.
- **Các phương pháp**:
    - **Top-down**: test giao diện/ UI trước, dùng stubs cho các phần dưới; tốt khi UI là critical nhưng làm chậm kiểm thử các controller thấp.
    - **Bottom-up**: test các controller/low-level trước, dùng drivers; tốt khi controller quan trọng nhưng UI được test muộn.
    - **Sandwich**: test top và bottom song song (stubs + drivers), sau đó ghép lại; hiệu quả về thời gian nhưng tốn công viết nhiều stubs/drivers.
- **Regression**: khi tích hợp thêm subsystem mới, chạy lại một tập test trước đó (subset) để phát hiện regressions.

System Testing Types and Pilot Testing

- **System testing** kiểm tra hệ thống như một tổng thể: **functional testing**, **performance**, **security**, **timing**, **recovery**, **volume**, v.v. — đảm bảo hệ thống thỏa yêu cầu chức năng và phi chức năng.
- **Pilot testing**:
    - **Alpha**: mời người dùng thử trong môi trường developer (controlled).
    - **Beta**: phát hành cho người dùng thật trên máy của họ; phổ biến trong game để thu feedback.

Acceptance Testing — cách suy ra test

- **Mục tiêu**: chứng minh với khách hàng rằng chức năng và ràng buộc đã được thực hiện.
- **Quy trình rút test từ yêu cầu**: nhóm các yêu cầu liên quan; loại bỏ yêu cầu không thể kiểm thử; bổ sung yêu cầu thu thập từ use cases/domain; cho mỗi yêu cầu tạo **evaluation scenario** minh chứng đạt yêu cầu.

Tóm tắt ngắn Lecture 7-3 Acceptance Testing Example

Lecture 7-3 minh họa cách chuyển yêu cầu thành **acceptance tests** có thể trình diễn trước khách hàng — tức là những kịch bản mà hệ thống phải **demonstrate** (thể hiện) được. Bài giảng dùng ví dụ hệ thống đăng ký học của ASU để: (1) trích xuất yêu cầu từ prompt, (2) loại bỏ những câu không testable, (3) viết lại yêu cầu theo dạng có thể kiểm thử, và (4) định nghĩa các kịch bản “demonstrate” để khách hàng chạy thử.

Cách rút acceptance tests từ yêu cầu — bước theo thứ tự

- **Thu thập mọi yêu cầu liên quan** từ prompt, use cases, và tài liệu hệ thống.
- **Loại bỏ hoặc chuyển hóa những câu không testable** (ví dụ: “student decides which courses to take” — không test được vì đó là hành vi con người).
- **Viết lại mỗi yêu cầu theo ngôn ngữ hệ thống**: dùng cấu trúc “the system must be able to …” hoặc “the system must produce …” để rõ ràng mục tiêu kiểm thử.

- **Định nghĩa acceptance scenario bằng từ khóa demonstrate**: mô tả bước thực hiện mà khách hàng sẽ chạy để chứng minh hệ thống đáp ứng yêu cầu.

Ví dụ acceptance tests cho hệ thống đăng ký học (ASU)

Lưu ý: mỗi test phải có *setup*, *bước thực hiện*, *kết quả mong đợi*, và *ai sẽ demo* (client hoặc tester).

- **Demo 1 Produce course catalog for term**
  - **Setup**: term = Fall 20XX; course offerings đã được nạp.
  - **Bước**: client yêu cầu course catalog cho term.
  - **Expected**: hệ thống hiển thị danh sách course offerings cho term; số lượng và mã môn khớp dữ liệu nguồn.
- **Demo 2 Select up to 4 primary courses**
  - **Setup**: student account có quyền đăng ký; course list có ≥4 khóa.
  - **Bước**: client chọn 4 khóa chính.
  - **Expected**: hệ thống chấp nhận 4 lựa chọn; nếu cố chọn khóa thứ 5 thì hiển thị thông báo lỗi và không cho thêm.
- **Demo 3 Select up to 2 alternative courses**
  - **Setup**: như trên.
  - **Bước**: client thêm 2 alternative courses.
  - **Expected**: hệ thống lưu alternative courses; không cho thêm quá 2.
- **Demo 4 Operational end-to-end registration**
  - **Setup**: student có đủ điều kiện, course không full.
  - **Bước**: client thực hiện đăng ký và xác nhận.
  - **Expected**: hệ thống cập nhật trạng thái đăng ký, gửi xác nhận, và hiển thị trong transcript của student.
- **Demo 5 Client acceptance runbook**
  - **Setup**: chuẩn bị dữ liệu demo, account client, checklist.
  - **Bước**: client chạy toàn bộ kịch bản acceptance trước mặt đại diện dự án.
  - **Expected**: mọi kịch bản “demonstrate” pass; nếu có fail, ghi lại bug và quyết định tiếp theo.

Đánh giá test và tiêu chí dừng

- **So sánh kết quả với test plan**: đánh giá pass/fail theo mục tiêu đã định.
- **Metrics gợi ý**: coverage của acceptance scenarios, error rate (tỷ lệ lỗi trên số test), số defects critical.
- **Quy tắc dừng**: xác định trước ngưỡng (ví dụ pass ≥ 99% hoặc error rate ≤ X) và điều kiện kinh doanh (ví dụ phần chức năng quan trọng phải 100% pass).

Bài tập thực hành nhanh và bước tiếp theo

- **Bài tập 1**: Từ prompt hệ thống bạn đang học, liệt kê 6 yêu cầu, chuyển 4 trong đó thành acceptance tests theo mẫu “demonstrate”, kèm setup và expected.
- **Bài tập 2**: Viết runbook ngắn (5–8 bước) để client demo Demo 2 và Demo 4 ở trên.
- **Bước tiếp theo tôi có thể làm ngay**:

1. **Viết chi tiết một acceptance test** (kịch bản, dữ liệu, checklist) cho bất kỳ yêu cầu nào bạn chọn.
2. **Tạo runbook demo** sẵn sàng để trình bày trước khách hàng.
3. **Sinh 8 câu hỏi luyện tập** kèm đáp án ngắn về Acceptance Testing.

Mooc 4

Bạn là một giảng viên giống như giảng viên của khóa học này coursera là một chuyên gia trong lĩnh vực Software Engineering: Software Design and Project Management hãy giảng cho tôi về những gì đang hiển thị và chương tôi đang học

Bạn đang học **Lecture 1-1: Introduction to System Design and Analysis** trong khóa *Software Engineering: Software Design and Project Management* trên Coursera. Đây là phần mở đầu của **Module 1: System Analysis and Design**. Nội dung chính của bài giảng này xoay quanh việc chuyển đổi từ **tài liệu yêu cầu (SRS – Software Requirement Specification)** sang một **mô hình thiết kế** có thể dùng để triển khai phần mềm.

🎯 Mục tiêu học tập

- Hiểu **vai trò và tầm quan trọng** của phân tích và thiết kế hệ thống trong phát triển phần mềm.
- Nắm được **các hoạt động chính** diễn ra trong giai đoạn này.
- Biết cách **xác định mục tiêu thiết kế** và xử lý các yêu cầu phi chức năng.
- Làm quen với **các mẫu kiến trúc (architectural patterns)** và **mẫu thiết kế (design patterns)**, cũng như khi nào nên áp dụng chúng.

🛠️ Nội dung cốt lõi

1. **Phân tích và thiết kế trước khi lập trình**
    - Không thể nhảy ngay từ yêu cầu sang code, vì use case model chưa đủ chi tiết.
    - Cần loại bỏ sự mơ hồ, dư thừa và mô tả yêu cầu bằng "ngôn ngữ của lập trình viên".
2. **Hai giai đoạn thiết kế hệ thống**
    - **Giai đoạn 1:** Xây dựng "skeleton" – kiến trúc phần mềm làm nền tảng ổn định.
    - **Giai đoạn 2:** Phát triển chi tiết – bổ sung các thành phần xung quanh skeleton để có hệ thống hoàn chỉnh.
3. **Xem xét môi trường triển khai**
    - Ví dụ: hệ thống phải chạy trên nhiều hệ điều hành → dùng **Bridge Design Pattern** để quản lý file manager, giúp dễ bảo trì và mở rộng.
4. **Tầm quan trọng của mô hình thiết kế**
    - Giúp mô tả hệ thống một cách chính xác, dễ hiểu, dễ bảo trì.
    - Tạo "blueprint" cho việc lập trình, giảm rủi ro khi triển khai.

📌 Ý nghĩa thực tiễn

Trong thực tế, nếu bạn phát triển một ứng dụng thương mại điện tử:

- **Phân tích hệ thống** sẽ giúp bạn xác định các use case như "đặt hàng", "thanh toán", "quản lý kho".
- **Thiết kế hệ thống** sẽ chuyển các use case này thành kiến trúc (ví dụ: microservices cho thanh toán, quản lý kho) và áp dụng mẫu thiết kế (ví dụ: Bridge để hỗ trợ nhiều cổng thanh toán).

🔑 Nội dung chính

1. **Subsystems và Information Hiding**
    - Hệ thống lớn được chia nhỏ thành nhiều subsystem.
    - Mỗi subsystem cung cấp dịch vụ riêng và che giấu chi tiết triển khai (information hiding).
    - Người dùng subsystem chỉ cần biết cách sử dụng qua interface, không cần biết bên trong.
2. **Layered Architecture (Kiến trúc phân lớp)**
    - **Closed Layer:** mỗi lớp chỉ phụ thuộc vào lớp ngay dưới nó.
    - **Open Layer:** lớp có thể phụ thuộc vào bất kỳ lớp nào bên dưới.
    - Thực tế thường có 3–5 lớp: UI ở trên cùng, controller ở giữa, và các subsystem lõi ở dưới.
3. **Đáp ứng yêu cầu phi chức năng (Non-functional requirements)**
    - **Performance:** ít subsystem, tập trung chức năng quan trọng.
    - **Security:** nhiều lớp, closed layer để bảo vệ subsystem quan trọng.
    - **Safety:** gom các chức năng an toàn vào subsystem riêng.
    - **Availability:** thêm redundancy để hệ thống luôn sẵn sàng.
    - **Maintainability:** subsystem nhỏ, độc lập, dễ thay đổi.
4. **Các Architectural Patterns phổ biến**
    - **Multi-layer pattern:** phân lớp hệ thống.
    - **Repository pattern:** nhiều client truy cập dữ liệu qua một repository (ví dụ DBMS).
    - **Client-Server pattern:** mô hình web app với client, server, database.
    - **Broker pattern:** dùng proxy và broker để truy cập tài nguyên từ xa.
    - **Transaction Processing pattern:** quản lý và phân phối transaction (ví dụ database engine).
    - **Pipe-and-Filter pattern:** dữ liệu đi qua chuỗi filter (ví dụ xử lý ảnh).
    - **MVC pattern:** tách biệt Model (dữ liệu), View (UI), Controller (logic).

📌 Ý nghĩa thực tiễn

Ví dụ: khi xây dựng một **ứng dụng ngân hàng trực tuyến**

- **Repository pattern**: quản lý dữ liệu khách hàng và giao dịch.
- **Client-Server pattern**: khách hàng dùng trình duyệt → server → database.
- **MVC pattern**: UI hiển thị số dư (View), dữ liệu tài khoản (Model), logic giao dịch (Controller).
- **Security requirement**: dùng kiến trúc phân lớp, subsystem quan trọng nằm sâu bên trong, chỉ truy cập qua nhiều lớp kiểm soát.

🔑 Nội dung chính

1. **Use Case là gì?**
    - Là mô tả hành vi của hệ thống khi tương tác với tác nhân (actor).
    - Tập trung vào *"hệ thống làm gì"* thay vì *"hệ thống được xây dựng như thế nào"*.
2. **Phân tích Use Case**

    - Xác định các actor (người dùng, hệ thống khác).
    - Xác định các use case chính (ví dụ: đăng nhập, đặt hàng, thanh toán).
    - Vẽ sơ đồ use case để minh họa mối quan hệ giữa actor và hệ thống.
3. **Mục tiêu của Use Case Analysis**
    - Làm rõ yêu cầu chức năng.
    - Giúp nhóm phát triển và khách hàng có cùng ngôn ngữ mô tả.
    - Chuẩn bị cho bước thiết kế chi tiết (class diagram, sequence diagram).
4. **Các bước điển hình**
    - Bắt đầu từ yêu cầu trong SRS.
    - Chuyển thành sơ đồ use case.
    - Phân tích chi tiết từng use case bằng kịch bản (scenario).
    - Liên kết use case với các yêu cầu phi chức năng (ví dụ: bảo mật, hiệu năng).

📌 Ý nghĩa thực tiễn

Ví dụ: trong một **hệ thống thương mại điện tử**

- Actor: khách hàng, quản trị viên, hệ thống thanh toán.
- Use case: “Đăng ký tài khoản”, “Đặt hàng”, “Thanh toán”, “Quản lý kho”.
- Phân tích: từ use case “Thanh toán” → xác định các bước: chọn phương thức, xác nhận đơn hàng, xử lý giao dịch.
- Kết quả: giúp thiết kế sequence diagram cho quy trình thanh toán, đảm bảo logic rõ ràng.

🔑 Nội dung chính

1. **Ba loại lớp trong thiết kế**
    - **Boundary classes**: liên quan đến công nghệ giao diện người dùng (UI).
    - **Entity classes**: quản lý dữ liệu và công nghệ lưu trữ.
    - **Control classes**: điều khiển hành vi của hệ thống, xử lý logic, phân phối, hiệu năng, và giao dịch.
2. **Các vấn đề cần xem xét khi thiết kế lớp**
    - Có cần lớp riêng cho mỗi node trong hệ thống phân tán không?
    - Có nên gộp lớp boundary để tăng hiệu năng?
    - Có cần quản lý transaction trong hệ thống?
3. **Tái sử dụng và đặc tả lớp**
    - Tận dụng thư viện hoặc design patterns có sẵn.
    - Mỗi lớp cần được đặc tả rõ: thuộc tính, quan hệ, phương thức, kiểu dữ liệu, phạm vi truy cập, và ràng buộc.
4. **Active classes**
    - Là lớp có **thread riêng** để kiểm soát hành vi.
    - Thường là control hoặc boundary classes.
5. **Tối ưu hóa mô hình thiết kế**
    - Dùng kế thừa để tăng tái sử dụng.
    - Loại bỏ vòng lặp trong sơ đồ lớp.
    - Gộp hoặc tách lớp để tránh dư thừa.
    - Sử dụng cache hoặc trì hoãn tính toán khi cần.
6. **Cohesion và Coupling**

    - **Cohesion (độ gắn kết):** mỗi lớp chỉ nên làm một việc duy nhất → tăng tính dễ hiểu và bảo trì.
    - **Coupling (độ phụ thuộc):** giảm số lượng và loại phụ thuộc giữa các lớp → hệ thống dễ mở rộng và cập nhật.

📌 Ý nghĩa thực tiễn

Ví dụ: trong một **ứng dụng thương mại điện tử**

- **Boundary class:** `ShoppingCartUI` hiển thị giỏ hàng cho người dùng.
- **Entity class:** `Product` lưu thông tin sản phẩm, `Order` lưu thông tin đơn hàng.
- **Control class:** `OrderController` xử lý logic đặt hàng, kiểm tra tồn kho, và gọi API thanh toán.
- **Active class:** `PaymentProcessor` chạy thread riêng để xử lý giao dịch.
- Thiết kế tốt sẽ đảm bảo mỗi lớp có **một trách nhiệm duy nhất**, và các lớp ít phụ thuộc vào nhau.

🔑 Nội dung chính

1. **Khái niệm cơ bản**
    - Biểu đồ trạng thái mô tả **các trạng thái** mà một đối tượng có thể tồn tại và **các chuyển đổi (transition)** giữa chúng.
    - Được vẽ dưới dạng **đồ thị có hướng**: node là trạng thái, arc là sự kiện gây ra thay đổi trạng thái.
2. **Ví dụ minh họa: Tài khoản ngân hàng**
    - Trạng thái: *in credit*, *overdrawn*, *frozen*, *closed*.
    - Chuyển đổi:
        - Khi số dư < 0 → từ *in credit* sang *overdrawn*.
        - Sau 3 tháng âm → sang *frozen*.
        - Gọi thao tác *close account* → sang *closed*.
        - Sau 5 năm đóng → sang *final state*.
3. **Các trạng thái đặc biệt**
    - **Initial state**: ký hiệu bằng chấm đen, khi đối tượng được tạo ra.
    - **Final state**: chấm đen trong vòng tròn, khi đối tượng bị hủy.
4. **Đặc điểm của trạng thái**
    - Trạng thái có **thời gian tồn tại** (duration).
    - Được xác định bởi giá trị thuộc tính hoặc liên kết với đối tượng khác (ví dụ: bản sao phim được liên kết với khách hàng → trạng thái *reserved*).
5. **Chuyển đổi (Transition)**
    - Là sự thay đổi từ trạng thái nguồn sang trạng thái đích.
    - Có ba thành phần: **event trigger**, **guard condition**, **effect list**.
    - Ví dụ:
        - Event trigger: gọi hàm *deposit()* hoặc *withdraw()*.
        - Guard condition: số dư < 0.
        - Effect list: gửi thông báo cho quản lý.
6. **Các loại sự kiện (Events)**
    - **Call event**: gọi một phương thức (deposit, withdraw).

    - **Change event**: điều kiện thay đổi (balance < 0).
    - **Time event**: sau một khoảng thời gian (3 tháng).
    - **Signal event**: nhận tín hiệu bất đồng bộ từ đối tượng khác.
7. **Hoạt động trong trạng thái**
    - **Entry activity**: hành động khi vào trạng thái (ví dụ: set echo = *).
    - **Do activity**: hành động trong trạng thái (ví dụ: nhập mật khẩu).
    - **Exit activity**: hành động khi rời trạng thái (ví dụ: reset echo).

📌 Ý nghĩa thực tiễn

Ví dụ: trong một **ứng dụng đăng nhập**

- Trạng thái: *Enter Password*, *Authenticated*, *Locked*.
- Entry activity: hiển thị ô nhập mật khẩu.
- Do activity: người dùng nhập ký tự.
- Exit activity: xóa mật khẩu khỏi bộ nhớ khi thoát.
- Transition: nhập sai 3 lần → sang *Locked*.

🔑 Nội dung chính

1. **Các trạng thái của một lớp học (class section)**
    - **Available**: còn chỗ trống, số sinh viên ban đầu = 0.
    - **Full**: khi số sinh viên đạt 40.
    - **Offered**: khi kết thúc đăng ký và số sinh viên ≥ 10.
    - **Cancelled**: khi kết thúc đăng ký nhưng số sinh viên < 10.
    - **Final/Destroyed**: khi lớp bị hủy hoàn toàn.
2. **Các sự kiện và chuyển đổi (Transitions)**
    - **Add student**: từ *Available* → tăng số lượng sinh viên.
    - **Drop student**: từ *Full* → quay lại *Available*.
    - **Registration closed**:
        - Nếu ≥ 10 sinh viên → sang *Offered*.
        - Nếu < 10 sinh viên → sang *Cancelled*.
    - **Cancel section**: hủy lớp, unregister sinh viên, xóa lịch học.
3. **Hoạt động trong trạng thái (Activities)**
    - Trong *Available*: có thể thêm hoặc xóa sinh viên.
    - Trong *Cancelled*: phải hủy đăng ký toàn bộ sinh viên, xóa lịch, xóa lớp.
4. **Composite State Machine Diagram**
    - **Sequential composite state**: một trạng thái chứa nhiều trạng thái con, ví dụ trạng thái *Forward gear* trong xe hơi có các substate: số 1, số 2, số 3.
    - **Concurrent composite state**: một trạng thái có nhiều vùng hoạt động song song, ví dụ trạng thái *Incomplete course* yêu cầu hoàn thành đồng thời lab, project, và exam.
    - **Synchronization bar**: đảm bảo các hoạt động song song bắt đầu/kết thúc cùng lúc.
5. **Khi nào cần vẽ state machine diagram?**
    - Không cần cho mọi lớp trong hệ thống.
    - Chỉ nên vẽ cho những lớp có **hành vi động phức tạp** để làm rõ logic.

📌 Ý nghĩa thực tiễn

Ví dụ: trong một **hệ thống đăng ký môn học**

- Khi sinh viên thêm môn → lớp chuyển từ *Available* sang *Full* nếu đạt giới hạn.
- Khi hết hạn đăng ký → lớp được *Offered* nếu đủ sinh viên, hoặc *Cancelled* nếu không đủ.
- Điều này giúp quản lý vòng đời của lớp học rõ ràng, tránh lỗi logic khi triển khai.

🔑 Nội dung chính

1. **Khái niệm Design Pattern**
    - Là **một mô tả hoặc khuôn mẫu** để giải quyết một vấn đề thiết kế phổ biến.
    - Không phải là code hoàn chỉnh, mà là **template** có thể áp dụng trong nhiều tình huống.
    - Thường được mô tả bằng **class diagram**, thể hiện quan hệ và tương tác giữa các lớp/đối tượng.
2. **Đặc điểm**
    - Dùng nhiều **inheritance (kế thừa)** và **delegation (ủy quyền)**.
    - Giúp lập trình viên mới học theo các giải pháp đã thành công, hành xử giống chuyên gia.
    - Có **pattern catalog** – tập hợp các mẫu thiết kế được ghi nhận và dùng trong nhiều ngữ cảnh.
3. **Ví dụ minh họa (Chess analogy)**
    - Muốn trở thành kỳ thủ giỏi, bạn phải học luật cơ bản, nguyên tắc chiến lược, và quan sát ván cờ của các cao thủ.
    - Tương tự, để trở thành chuyên gia thiết kế phần mềm, bạn phải học thuật toán, cấu trúc dữ liệu, ngôn ngữ lập trình, và quan trọng là **nghiên cứu các mẫu thiết kế của chuyên gia**.
4. **Ý nghĩa thực tiễn**
    - Giúp giải quyết **yêu cầu phi chức năng** như hiệu năng, bảo mật, khả năng mở rộng.
    - Tăng khả năng **tái sử dụng** và **bảo trì** hệ thống.
    - Là nền tảng để học các mẫu cụ thể như Singleton, Observer, Factory, MVC…

📌 Ví dụ thực tế

Trong một **ứng dụng thương mại điện tử**:

- **Singleton pattern**: đảm bảo chỉ có một đối tượng `DatabaseConnection` để quản lý truy cập dữ liệu.
- **Observer pattern**: khi khách hàng đặt hàng, hệ thống thông báo cho kho, bộ phận giao hàng, và kế toán.
- **Factory pattern**: tạo đối tượng `PaymentMethod` (Visa, PayPal, Momo) mà không cần biết chi tiết triển khai.

🔑 Nội dung chính

1. **Khái niệm Strategy Pattern**
    - Cho phép định nghĩa một **họ các thuật toán** (strategies), đóng gói chúng, và làm cho chúng có thể thay thế cho nhau.
    - Tách biệt **thuật toán** khỏi **ngữ cảnh sử dụng**.
    - Giúp hệ thống dễ mở rộng, dễ bảo trì, và giảm sự phụ thuộc cứng giữa các lớp.
2. **Cấu trúc UML cơ bản**
    - **Context**: lớp sử dụng strategy, chứa tham chiếu đến interface strategy.
    - **Strategy (interface)**: định nghĩa phương thức chung cho các thuật toán.
    - **Concrete Strategies**: các lớp triển khai thuật toán cụ thể.
    - Context có thể thay đổi strategy trong runtime mà không cần sửa code.
3. **Ví dụ minh họa**
    - **Thanh toán trong thương mại điện tử**:
        - Strategy: `PaymentMethod` (interface).
        - Concrete Strategies: `CreditCardPayment`, `PayPalPayment`, `MomoPayment`.
        - Context: `Checkout` class, có thể chọn phương thức thanh toán tùy người dùng.
    - **Sắp xếp dữ liệu**:
        - Strategy: `SortAlgorithm`.
        - Concrete Strategies: `QuickSort`, `MergeSort`, `BubbleSort`.
        - Context: `DataSorter`, có thể thay đổi thuật toán tùy kích thước dữ liệu.
4. **Ưu điểm**
    - Dễ mở rộng thêm thuật toán mới.
    - Giảm sự trùng lặp code.
    - Tăng tính linh hoạt khi thay đổi hành vi.
5. **Nhược điểm**
    - Tăng số lượng lớp trong hệ thống.
    - Người dùng cần hiểu rõ để chọn strategy phù hợp.

📌 Ý nghĩa thực tiễn

Trong một **ứng dụng ngân hàng trực tuyến**:

- Khi khách hàng chuyển tiền, hệ thống có thể chọn strategy khác nhau: `DomesticTransfer`, `InternationalTransfer`, `CryptoTransfer`.
- Tất cả đều tuân theo interface `TransferStrategy`, nhưng logic xử lý khác nhau.
- Điều này giúp hệ thống dễ mở rộng khi thêm phương thức chuyển tiền mới mà không ảnh hưởng đến code hiện tại.

🔑 Nội dung chính

1. **Khái niệm cơ bản**
    - Observer Pattern mô tả quan hệ **một-nhiều (one-to-many)** giữa **Subject** và **Observers**.

    - Khi Subject thay đổi trạng thái, nó sẽ **notify** tất cả các Observer đã đăng ký để chúng tự cập nhật.
    - Các đối tượng được **loosely coupled**: Subject chỉ biết Observer qua interface, không phụ thuộc vào triển khai cụ thể.
2. **Ví dụ minh họa: Spreadsheet Application**
    - Subject: dữ liệu trong bảng tính.
    - Observers: các biểu đồ (charts) dựa trên dữ liệu đó.
    - Khi dữ liệu thay đổi → tất cả biểu đồ được cập nhật tự động.
    - Một Observer có thể subscribe hoặc unsubscribe bất kỳ lúc nào.
3. **Cấu trúc UML**
    - **Subject interface**: định nghĩa phương thức để đăng ký, hủy đăng ký, và notify observers.
    - **Concrete Subject**: triển khai Subject, giữ trạng thái dữ liệu và danh sách observers.
    - **Observer interface**: định nghĩa phương thức update().
    - **Concrete Observers**: triển khai Observer, nhận thông báo và cập nhật trạng thái.
4. **Ưu điểm**
    - Giảm sự phụ thuộc chặt chẽ giữa các đối tượng.
    - Dễ mở rộng: thêm Observer mới mà không cần thay đổi Subject.
    - Hỗ trợ cơ chế broadcast communication.
5. **Thách thức khi triển khai**
    - Quản lý protocol thông báo sự kiện: có phải mọi sự kiện đều gửi đến mọi Observer?
    - Cho phép Observer chọn lọc sự kiện để subscribe.
    - Xử lý hiệu năng khi có nhiều Observer cùng nhận thông báo.

📌 Ý nghĩa thực tiễn

- **Ứng dụng thương mại điện tử**: khi đơn hàng được xác nhận (Subject), hệ thống sẽ notify kho, bộ phận giao hàng, và kế toán (Observers).
- **Ứng dụng mạng xã hội**: khi người dùng đăng bài (Subject), tất cả follower (Observers) nhận thông báo.
- **Ứng dụng ngân hàng**: khi tài khoản thay đổi số dư, hệ thống notify các dịch vụ liên quan như cảnh báo SMS, email, và dashboard quản lý.

🔑 Nội dung chính

1. **Vấn đề trước khi có Mediator**
    - Trong hệ thống **smart home** ví dụ: alarm, calendar, sprinkler, coffee pot…
    - Nếu các thiết bị giao tiếp trực tiếp với nhau, mỗi thiết bị phải tự quản lý nhiều quy tắc (rules).
    - Điều này khiến việc bảo trì khó khăn vì không biết quy tắc nằm ở đâu (alarm, calendar hay sprinkler).
2. **Giải pháp Mediator Pattern**
    - Thay vì để các “colleagues” (thiết bị) giao tiếp trực tiếp, tất cả đều tương tác thông qua **Mediator**.
    - Mediator giữ toàn bộ quy tắc và điều phối hành vi.

    - Khi có sự kiện (ví dụ: alarm kêu), Mediator sẽ quyết định và thông báo cho coffee pot hoặc sprinkler.
3. **Cấu trúc UML**
    - **Mediator interface**: định nghĩa cách giao tiếp.
    - **Concrete Mediator**: triển khai logic điều phối.
    - **Colleague interface**: định nghĩa cách các đối tượng tham gia.
    - **Concrete Colleagues**: các đối tượng cụ thể (alarm, coffee pot, sprinkler).
    - Mỗi colleague có tham chiếu đến Mediator, Mediator có tham chiếu đến các colleague.
4. **Ứng dụng thực tế**
    - **GUI components**: khi chọn một option trong UI, Mediator sẽ bật/tắt các thành phần khác.
    - **Smart home**: alarm → Mediator → coffee pot, sprinkler.
    - **Chat system**: Mediator quản lý tin nhắn giữa nhiều người dùng thay vì họ gửi trực tiếp cho nhau.
5. **Ưu điểm**
    - Giảm coupling: các đối tượng không còn phụ thuộc trực tiếp vào nhau.
    - Dễ bảo trì: chỉ cần cập nhật logic trong Mediator.
    - Đơn giản hóa giao thức giữa các đối tượng.
    - Giảm nhu cầu subclassing: chỉ cần mở rộng Mediator khi thêm quy tắc mới.
6. **Nhược điểm**
    - Mediator có thể trở thành **"God object"** – quá phức tạp vì phải quản lý tất cả quy tắc.
    - Việc triển khai Mediator đôi khi khó khăn nếu hệ thống có nhiều logic phức tạp.

📌 Ý nghĩa thực tiễn

Ví dụ: trong một **ứng dụng đặt vé máy bay**

- Các module: thanh toán, kiểm tra chỗ ngồi, gửi email xác nhận.
- Nếu chúng giao tiếp trực tiếp → phức tạp.
- Mediator sẽ quản lý: khi thanh toán thành công → thông báo cho module chỗ ngồi và module email.

🔑 Nội dung chính

1. **Khái niệm Proxy Pattern**
    - Proxy là một lớp **đóng vai trò trung gian** giữa client và đối tượng thực (real subject).
    - Client không giao tiếp trực tiếp với đối tượng thực, mà thông qua proxy.
    - Proxy có thể thêm logic bổ sung trước hoặc sau khi gọi đối tượng thực.
2. **Các loại Proxy phổ biến**
    - **Remote Proxy**: đại diện cho đối tượng nằm trên máy khác (ví dụ: gọi dịch vụ qua mạng).
    - **Virtual Proxy**: trì hoãn việc khởi tạo đối tượng tốn kém cho đến khi thật sự cần.

    - **Protection Proxy**: kiểm soát quyền truy cập, đảm bảo chỉ những client hợp lệ mới được gọi đối tượng.
    - **Cache Proxy**: lưu kết quả để giảm chi phí truy cập đối tượng thực.
3. **Cấu trúc UML**
    - **Subject interface**: định nghĩa phương thức chung cho proxy và real subject.
    - **Real Subject**: đối tượng thực hiện công việc chính.
    - **Proxy**: triển khai Subject, giữ tham chiếu đến Real Subject, và có thể thêm logic kiểm soát.
    - **Client**: chỉ biết đến Subject, không cần biết đang làm việc với Proxy hay Real Subject.
4. **Ví dụ minh họa**
    - **Hệ thống quản lý ảnh**:
        - Real Subject: `HighResolutionImage`.
        - Proxy: `ImageProxy`, chỉ tải ảnh thật khi người dùng yêu cầu hiển thị.
    - **Hệ thống bảo mật**:
        - Real Subject: `SensitiveData`.
        - Proxy: `AccessControlProxy`, kiểm tra quyền trước khi cho phép truy cập dữ liệu.
5. **Ưu điểm**
    - Giảm chi phí khởi tạo đối tượng nặng.
    - Tăng bảo mật và kiểm soát truy cập.
    - Cho phép thêm chức năng mà không cần thay đổi đối tượng thực.
6. **Nhược điểm**
    - Tăng độ phức tạp hệ thống.
    - Có thể gây chậm trễ nếu proxy xử lý nhiều logic bổ sung.

📌 Ý nghĩa thực tiễn

Ví dụ: trong một **ứng dụng thương mại điện tử**

- Khi khách hàng xem ảnh sản phẩm, hệ thống dùng **Virtual Proxy** để chỉ tải ảnh độ phân giải cao khi người dùng phóng to.
- Khi truy cập thông tin thanh toán, dùng **Protection Proxy** để đảm bảo chỉ admin mới xem được dữ liệu nhạy cảm.

🔑 Bridge Pattern

1. **Vấn đề đặt ra**
    - Ví dụ: bạn có nhiều loại *Window* (Icon Window, Application Window) và mỗi loại lại phải chạy trên nhiều hệ điều hành (Windows OS, MacOS).
    - Nếu kết hợp trực tiếp, bạn sẽ phải tạo rất nhiều lớp (IconWindowOnWindows, IconWindowOnMac, ApplicationWindowOnWindows, ApplicationWindowOnMac…). → **bùng nổ số lượng lớp**.
2. **Giải pháp Bridge**
    - Tách **abstraction** (Window, IconWindow, ApplicationWindow) khỏi **implementation** (WindowsOSImplementation, MacOSImplementation).
    - Mỗi Window giữ một tham chiếu đến interface của implementation.

    - Khi cần vẽ Icon Window trên Windows OS → Window gọi implementation tương ứng.
3. **Ý nghĩa**
    - Giảm số lượng lớp cần thiết.
    - Dễ mở rộng: thêm loại Window hoặc thêm hệ điều hành mới mà không ảnh hưởng đến phần còn lại.
    - Tăng tính linh hoạt và khả năng bảo trì.

🔑 Singleton Pattern

1. **Khái niệm**
    - Đảm bảo chỉ có **một instance duy nhất** của một lớp trong toàn bộ hệ thống.
    - Cung cấp một điểm truy cập toàn cục (global access point).
2. **Ví dụ thực tế**
    - **Thread pool**: chỉ cần một đối tượng quản lý các thread.
    - **Cache**: một bộ nhớ đệm dùng chung.
    - **Dialog box**: chỉ hiển thị một cửa sổ thông báo tại một thời điểm.
3. **Cấu trúc UML**
    - Lớp có một biến static để lưu instance duy nhất.
    - Phương thức `getInstance()` trả về instance này, tạo mới nếu chưa tồn tại.
4. **Ý nghĩa**
    - Đảm bảo tính nhất quán dữ liệu.
    - Tiết kiệm tài nguyên.
    - Dễ quản lý các đối tượng dùng chung.

📌 Tóm lại

- **Bridge Pattern**: giải quyết vấn đề kết hợp abstraction và implementation, tránh bùng nổ số lượng lớp.
- **Singleton Pattern**: đảm bảo chỉ có một instance duy nhất cho những đối tượng dùng chung.

Ví dụ: trong một **ứng dụng thương mại điện tử**

- Dùng **Bridge Pattern** để hiển thị giao diện trên nhiều nền tảng (Web, iOS, Android).
- Dùng **Singleton Pattern** cho `DatabaseConnection` để tất cả module dùng chung một kết nối.

🔑 Nội dung chính

1. **Vấn đề trước khi có Factory**
    - Khi khởi tạo đối tượng, bạn phải chỉ rõ subtype (ví dụ: MallardDuck, DecoyDuck, RubberDuck).
    - Nếu thêm loại mới, bạn phải sửa code gốc → vi phạm nguyên tắc “closed for modification”.
2. **Giải pháp Factory Pattern**
    - Tách phần khởi tạo đối tượng ra khỏi logic chính.

    - Đưa vào một **Factory class** chuyên trách việc tạo đối tượng dựa trên tham số (ví dụ: loại pizza, loại duck).
    - Khi cần thêm loại mới, chỉ cần mở rộng Factory, không phải sửa code gốc.
3. **Ví dụ minh họa: Pizza Shop**
    - Trước đây: code phải khởi tạo trực tiếp `CheesePizza`, `GreekPizza`, `VeggiePizza`.
    - Sau khi áp dụng Factory: `PizzaFactory` nhận tham số loại pizza và trả về đối tượng tương ứng.
    - Pizza Store chỉ gọi Factory để tạo pizza, rồi thực hiện các bước chuẩn bị, nướng, cắt, đóng hộp.
4. **Cấu trúc UML**
    - **Product interface**: định nghĩa chung cho sản phẩm (Pizza).
    - **Concrete Products**: các loại pizza cụ thể.
    - **Creator interface (Factory)**: định nghĩa phương thức tạo sản phẩm.
    - **Concrete Factory**: triển khai việc tạo sản phẩm cụ thể.
5. **Nguyên tắc thiết kế áp dụng**
    - **Open-Closed Principle**: lớp mở rộng được nhưng hạn chế sửa đổi.
    - **Encapsulation of variation**: tách phần thay đổi (loại sản phẩm) khỏi phần cố định (quy trình xử lý).
6. **Ưu điểm**
    - Dễ mở rộng khi thêm loại sản phẩm mới.
    - Giảm sự phụ thuộc giữa client và lớp cụ thể.
    - Tăng khả năng tái sử dụng và bảo trì.
7. **Nhược điểm**
    - Tăng số lượng lớp trong hệ thống.
    - Có thể phức tạp hơn nếu sản phẩm đa dạng.

📌 Ý nghĩa thực tiễn

Ví dụ: trong một **ứng dụng thương mại điện tử**

- Khi khách hàng chọn phương thức thanh toán, Factory Pattern có thể tạo ra `CreditCardPayment`, `PayPalPayment`, hoặc `MomoPayment`.
- Khi thêm phương thức mới (ví dụ: `CryptoPayment`), chỉ cần mở rộng Factory mà không phải sửa code gốc.

🔑 Nội dung chính

1. **Khái niệm Anti-Pattern**
    - Design patterns: giải pháp tốt, nên áp dụng.
    - Anti-patterns: giải pháp xấu, cần tránh.
    - Mục tiêu: nhận diện và tránh lặp lại những sai lầm trong thiết kế/phát triển.
2. **Các loại Anti-Pattern**
    - Liên quan đến phát triển (development).
    - Liên quan đến hướng đối tượng (object-oriented).
    - Liên quan đến tổ chức (organizational).
    - Liên quan đến miền ứng dụng (domain-specific).

3. **Ví dụ phổ biến trong phần mềm**
    - **Spaghetti Code**:
        - Code không có cấu trúc rõ ràng, khó mở rộng và bảo trì.
        - Triệu chứng: code viết vội để "chạy được", thiếu tài liệu, nhiều workaround, quan hệ giữa các đối tượng không rõ ràng.
        - Hậu quả: mất lợi thế của lập trình hướng đối tượng, bảo trì tốn kém, thường phải viết lại toàn bộ.
    - **God Class**:
        - Một lớp duy nhất chứa quá nhiều thuộc tính và phương thức.
        - Vi phạm nguyên tắc cohesion (mỗi lớp chỉ nên có một trách nhiệm).
        - Hậu quả: khó bảo trì, khó kiểm thử, dễ sinh lỗi khi sửa đổi.
4. **Nguyên nhân dẫn đến Anti-Pattern**
    - Yêu cầu không rõ ràng hoặc sai lệch.
    - Thiếu kiến trúc hoặc không tuân thủ kiến trúc.
    - Thiếu tài liệu, thiếu giao tiếp trong nhóm.
    - Lập trình viên làm việc đơn lẻ, không phối hợp.
5. **Hậu quả**
    - Mất lợi thế của thiết kế hướng đối tượng.
    - Hệ thống phức tạp, khó bảo trì, tốn chi phí.
    - Cuối cùng thường phải refactor hoặc viết lại.

📌 Ý nghĩa thực tiễn

Ví dụ:

- Trong một **ứng dụng quản lý sinh viên**, nếu tất cả logic (đăng ký môn học, tính điểm, quản lý học phí) đều nằm trong một lớp `StudentManager` → đó là **God Class**.
- Trong một **ứng dụng thương mại điện tử**, nếu code xử lý đơn hàng, thanh toán, kho hàng viết lẫn lộn, không có cấu trúc rõ ràng → đó là **Spaghetti Code**.

🔑 Nội dung chính

1. **Khái niệm SQA**
    - Là tập hợp các **thủ tục, kỹ thuật, công cụ** được áp dụng để đảm bảo sản phẩm phần mềm đáp ứng hoặc vượt qua các tiêu chuẩn đã định trước.
    - Không chỉ là kiểm thử, mà phải thực hiện trong **mọi giai đoạn của dự án**.
2. **Các hoạt động chính**
    - **Quality Assurance**: xác định và áp dụng các **tiêu chuẩn tổ chức** mà mọi thành viên phải tuân thủ.
    - **Quality Planning**: chọn và điều chỉnh các tiêu chuẩn phù hợp cho từng dự án cụ thể.
    - **Quality Control**: giám sát việc tuân thủ tiêu chuẩn trong suốt dự án, đảm bảo chất lượng được duy trì.
3. **Nguyên tắc quan trọng**
    - **Phát hiện lỗi sớm** → chi phí sửa chữa thấp.
    - **Phát hiện lỗi muộn** → chi phí sửa chữa cao.

    - Do đó, các hoạt động SQA đầu dự án sẽ **tiết kiệm chi phí** và nâng cao chất lượng.
4. **Bốn yếu tố "4P" trong chất lượng phần mềm**
    - **Project**: cần có kế hoạch dự án tốt ngay từ đầu.
    - **People**: đội ngũ phải được đào tạo và có kỹ năng.
    - **Process**: quy trình phát triển phải rõ ràng, có đo lường và phản hồi.
    - **Product**: sản phẩm phải đáp ứng các **quality attributes** (ví dụ: hiệu năng, bảo mật, khả năng bảo trì).
5. **Đo lường và cải tiến liên tục**
    - Thiết lập các **quality attributes** ngay từ đầu (design goals).
    - Theo dõi giá trị của các thuộc tính này trong suốt dự án.
    - Dùng kết quả từ dự án trước để cải thiện chất lượng cho dự án sau.

📌 Ý nghĩa thực tiễn

Ví dụ: trong một **ứng dụng ngân hàng trực tuyến**

- **Quality Assurance**: áp dụng tiêu chuẩn bảo mật PCI DSS.
- **Quality Planning**: chọn thêm tiêu chuẩn mã hóa dữ liệu cho dự án này.
- **Quality Control**: kiểm tra định kỳ để đảm bảo mọi module đều tuân thủ.
- **Quality Attributes**: tốc độ xử lý giao dịch < 2 giây, hệ thống uptime ≥ 99.9%.

🔑 Nội dung chính

1. **Thiết lập tiêu chuẩn (Standards)**
    - Mọi thành viên trong tổ chức phải tuân thủ các tiêu chuẩn chung.
    - Có hai loại:
        - **Product standards**: tập trung vào sản phẩm, xác định đặc tính mà mọi artifact phải có để đảm bảo chất lượng.
        - **Process standards**: tập trung vào quy trình, xác định cách thực hiện để đạt chất lượng.
    - Tiêu chuẩn giúp tránh sai sót, tạo khung để kiểm soát chất lượng, và đảm bảo tính liên tục giữa các dự án.
2. **Thiết lập chỉ số đo lường (Metrics)**
    - **Metric** là bất kỳ loại đo lường nào liên quan đến sản phẩm, quy trình hoặc artifact.
    - Giúp kiểm soát quá trình phát triển, dự đoán chất lượng sản phẩm, và ước lượng nguồn lực (thời gian, chi phí).
    - Ví dụ: **Cyclomatic Complexity** dùng để dự đoán khả năng bảo trì. Nếu độ phức tạp cao → hệ thống khó bảo trì.
3. **Các hoạt động bổ trợ khác**
    - Sử dụng công cụ tốt để phát triển.
    - Thực hiện **reviews** định kỳ để phát hiện lỗi sớm.
    - Quản lý cấu hình phần mềm (SCM) để theo dõi thay đổi.
    - Kiểm thử đầy đủ để giảm thiểu lỗi trong sản phẩm cuối cùng.
4. **Ý nghĩa của Standards và Metrics**
    - **Standards**: ghi lại best practices, tạo khung kiểm soát chất lượng, giảm công sức học lại khi bắt đầu dự án mới.

- **Metrics**: cung cấp dữ liệu định lượng để ra quyết định, thay vì chỉ dựa vào cảm tính.

📌 Ví dụ thực tế

Trong một **ứng dụng thương mại điện tử**:

- **Product standards**: mọi module phải có tài liệu thiết kế, code phải tuân thủ coding style.
- **Process standards**: mỗi pull request phải được review bởi ít nhất 2 người.
- **Metrics**: đo số lượng bug tìm thấy trong mỗi sprint, đo thời gian phản hồi API, đo cyclomatic complexity của các module quan trọng.

🔑 Nội dung chính

1. **Design Goals (Mục tiêu thiết kế)**
   - Là các **quality attributes** đặt ra từ đầu dự án: dễ hiểu (understandability), dễ bảo trì (maintainability), độ tin cậy (reliability), khả năng thích ứng (adaptability)…
   - Thường chỉ đánh giá được sau khi hệ thống hoàn thành, nhưng có thể dự đoán trong quá trình phát triển bằng cách dùng **internal attributes**.
2. **Mapping External → Internal Attributes**
   - External attributes (ví dụ: maintainability, usability) khó đo trực tiếp.
   - Chúng được liên kết với internal attributes có thể đo lường:
     - Maintainability ↔ số lượng tham số, cyclomatic complexity, số dòng code, số lượng thông báo lỗi.
     - Usability ↔ số lượng thông báo lỗi, độ dài tài liệu hướng dẫn.
3. **Đo lường độ phức tạp (Complexity Metrics)**
   - **Fan-in / Fan-out**:
     - Fan-in cao → coupling cao, khó bảo trì.
     - Fan-out cao → lớp gọi nhiều lớp khác, tăng độ phức tạp.
   - **Informational fan-in/out**: số lượng tham số và truy cập dữ liệu chia sẻ.
   - Công thức tính độ phức tạp: *component length* × *fan-in* × *fan-out*².
   - Được kiểm chứng trong hệ thống Unix, dự đoán tốt effort triển khai.
4. **Các chỉ số chất lượng khác**
   - **DSQI (Design Structure Quality Index)**: nếu gần 1 → thiết kế tốt; nếu thấp → cần cải thiện.
   - **SMI (Software Maturity Index)**: đo độ ổn định của hệ thống qua vòng đời. Nếu gần 1 → hệ thống ổn định.
   - **Halstead's Software Science**: dựa trên volume, difficulty, effort.
   - **McCabe's Cyclomatic Complexity**: đo số lượng đường đi độc lập trong code.
   - **Lines of Code, Identifier length, Nesting depth**: càng cao → càng khó hiểu và khó bảo trì.
5. **Quản lý lỗi (Defect Management)**
   - Phải chứng minh chương trình và đặc tả đúng.
   - Theo dõi, phân loại, xác định nguyên nhân lỗi.

- Áp dụng quy tắc **80-20**: 80% lỗi đến từ 20% nguyên nhân → tập trung sửa 20% này để cải thiện lớn.

📌 Ý nghĩa thực tiễn

Ví dụ: trong một **ứng dụng ngân hàng trực tuyến**

- Đặt **design goals**: tốc độ giao dịch < 2 giây, dễ bảo trì, bảo mật cao.
- Dùng **metrics**: cyclomatic complexity để kiểm soát độ phức tạp, DSQI để đánh giá thiết kế, SMI để theo dõi độ ổn định qua các phiên bản.
- Quản lý lỗi: tập trung vào module thanh toán (20% nguyên nhân) để giảm phần lớn lỗi hệ thống.

🔑 Nội dung chính

1. **Project Quality**
   - Thực hiện **reviews thường xuyên** (không chỉ code review mà cả yêu cầu, phân tích, thiết kế).
   - 50–60% lỗi thường xuất hiện ở giai đoạn yêu cầu và thiết kế.
   - Các **formal technical reviews** có thể phát hiện tới 75% lỗi sớm.
   - **Software Configuration Management (SCM)**: kiểm soát và giám sát mọi thay đổi trong dự án.
2. **Process Quality**
   - Phần mềm là sản phẩm **thiết kế, sáng tạo**, không phải sản xuất cơ học.
   - Quy trình phát triển phần mềm mang tính **tổ chức đặc thù**, không thể sao chép nguyên xi từ công ty khác.
   - **SEI-CMM (Capability Maturity Model)**: thang đo mức độ trưởng thành của quy trình phát triển:
     - Level 1: ad hoc, không có quản lý.
     - Level 2: có quản lý cơ bản (theo dõi thời gian, ngân sách).
     - Level 3: quy trình được định nghĩa, có đào tạo.
     - Level 4: có đo lường trong suốt dự án.
     - Level 5: có phản hồi để cải tiến liên tục.
3. **People Quality**
   - Con người thường quan trọng hơn cả quy trình.
   - **PCMM (People Capability Maturity Model)**: đánh giá và cải thiện kỹ năng nhân sự:
     - Level 1: không có đào tạo.
     - Level 2: đào tạo cơ bản (ví dụ: ngôn ngữ lập trình).
     - Level 3: định nghĩa, có kế hoạch phát triển nhân tài.
     - Level 4: quản lý, tập trung vào xây dựng đội nhóm.
     - Level 5: cải thiện liên tục kỹ năng cá nhân và nhóm, áp dụng best practices.
4. **Tóm tắt**
   - Tổ chức cần có **quality manual** (sổ tay chất lượng).
   - Mỗi dự án cần có **quality plan** (kế hoạch chất lượng).
   - Có **standards handbook** để mọi nhân viên tuân thủ.
   - Thiết lập **cơ chế giám sát** việc tuân thủ chất lượng.

    - Theo dõi **metrics** để phát hiện bất thường.
    - Testing quan trọng nhưng không đủ – chất lượng phải được xây dựng từ đầu.

📌 Ý nghĩa thực tiễn

Ví dụ: trong một **công ty phần mềm lớn**

- Nếu ở **CMM Level 3 trở lên**, công ty sẽ có quy trình rõ ràng, đào tạo nhân viên, và xây dựng đội nhóm tốt.
- Điều này giúp dự án ít lỗi hơn, dễ bảo trì, và nhân viên phát triển kỹ năng trong quá trình làm việc.

🔑 Nội dung chính

1. **Tầm quan trọng của Project Management**
    - Quản lý dự án yếu thường gây **thất bại nghiêm trọng** cho dự án phần mềm.
    - Project management phải được thực hiện **xuyên suốt vòng đời dự án**, không chỉ ở giai đoạn đầu.
2. **Nhiệm vụ chính của Project Manager**
    - Lập **development plan**: xác định thông tin về dự án, các tính năng cần có, nguồn lực, ngân sách, thời gian.
    - Lập kế hoạch với **thông tin chưa đầy đủ** (incomplete knowledge) → phải đưa ra ước lượng trước khi có dữ liệu thực tế.
    - Ước lượng: số lượng nhân sự cần thiết, số dòng code, chi phí, thời gian.
    - Quản lý dự án với **nguồn lực hạn chế**: thời gian, tiền bạc, kỹ năng.
3. **Các quyết định quan trọng trong kế hoạch**
    - Tính năng nào cần phát triển?
    - Nhiệm vụ nào phải thực hiện?
    - Mua hay tự xây dựng tính năng?
    - Nguồn lực cần thiết và mức độ nỗ lực.
4. **Nguyên tắc quản lý dự án**
    - **Plan the work and work the plan**: vừa lập kế hoạch vừa thực hiện, điều chỉnh liên tục.
    - **Project tracking and control**: theo dõi tiến độ, phát hiện sớm vấn đề, điều chỉnh để tránh thất bại.
5. **Ví dụ minh họa**
    - Video trong bài giảng cho thấy nhiều tình huống thất bại:
        - Ước lượng chi phí sai lệch.
        - Lập kế hoạch dựa trên giả định phi thực tế.
        - Trì hoãn liên tục, không hoàn thành đúng hạn.
        - Giao diện người dùng gây phản ứng tiêu cực.
    - Những tình huống này nhấn mạnh rằng **thiếu quản lý dự án tốt → dự án thất bại**.

📌 Ý nghĩa thực tiễn

Trong một **ứng dụng thương mại điện tử**:

- Nếu không có kế hoạch rõ ràng về số lượng sản phẩm, chi phí server, thời gian phát triển → dễ dẫn đến trễ tiến độ và vượt ngân sách.
- Một project manager giỏi sẽ liên tục theo dõi tiến độ, điều chỉnh nguồn lực, và đảm bảo dự án đi đúng hướng.

🔑 Nội dung chính

1. **Phạm vi dự án (Scope)**
    - Xác định rõ **những gì sẽ làm** và **những gì không làm** trong dự án.
    - SDP chính là tài liệu định nghĩa dự án.
2. **Deliverables (Sản phẩm bàn giao)**
    - **Client products**: code thực thi, hướng dẫn sử dụng.
    - **Process artifacts**: phân tích yêu cầu, tài liệu thiết kế, source code.
    - **Internal deliverables**: thư viện mã nguồn, makefiles.
    - **Services**: đào tạo, cài đặt, tùy chỉnh.
    - Deliverables ảnh hưởng trực tiếp đến **staffing và tổ chức nhóm**.
3. **Môi trường phát triển (Development Environment)**
    - Xác định phần cứng, phần mềm, công cụ phát triển.
    - Công cụ chỉ hiệu quả khi đi kèm quy trình tốt.
    - Cần đánh giá rủi ro khi áp dụng công cụ mới (chi phí, đào tạo).
4. **Work Breakdown Structure (WBS)**
    - Chia dự án thành **tasks và subtasks** theo mô hình cây.
    - Giúp dễ dàng phân công, theo dõi tiến độ, kiểm soát chi phí.
    - Granularity phải hợp lý: quá nhỏ khó quản lý, quá lớn khó kiểm soát.
5. **Staffing & Organization**
    - Xác định vai trò, trách nhiệm, số lượng nhân sự.
    - Tổ chức nhóm theo **modular structure** để giảm chi phí giao tiếp.
    - Mỗi subsystem cần có người phụ trách rõ ràng.
6. **Project Schedule**
    - Bao gồm **task ordering, estimates, milestones, deliverables, critical path**.
    - Có 3 mức độ: **master schedule**, **macro schedule** (quản lý ngày thường), **micro schedule** (quản lý nhóm).
    - Công cụ: **Gantt chart, PERT chart, burndown chart**.
7. **Estimating & Metrics**
    - Ước lượng dựa trên kinh nghiệm, dữ liệu lịch sử, mô hình.
    - Dùng **size-oriented metrics** (KLOC) hoặc **function-oriented metrics** (số input, output, file, interface).
    - Kỹ thuật: **Delphi technique**, **PERT estimation** (optimistic, most likely, pessimistic).
    - Thu thập metrics để theo dõi tiến độ và chất lượng.
8. **Risk Management & Budget**
    - Dự đoán rủi ro, đánh giá xác suất và tác động, lập kế hoạch dự phòng.
    - Lập **time-phased budget**: chi phí theo từng giai đoạn.
    - Manpower là chi phí chính, ngoài ra còn có training, license, hardware.
    - Nên có **10–15% dự phòng** để xử lý sự cố.

📌 Ý nghĩa thực tiễn

Ví dụ: trong một **dự án phát triển ứng dụng ngân hàng trực tuyến**

- Deliverables: app mobile, tài liệu hướng dẫn, báo cáo bảo mật.
- WBS: chia thành các module (đăng nhập, chuyển tiền, quản lý tài khoản).
- Schedule: dùng Gantt chart để theo dõi milestone (alpha, beta, release).
- Metrics: số lượng use case, số dòng code, cyclomatic complexity.
- Risk: rủi ro bảo mật, rủi ro trễ tiến độ do tích hợp hệ thống bên ngoài.

🔑 Nội dung chính

1. **Theo dõi tiến độ dự án**
    - Dự án thường bị trễ "mỗi ngày một chút" → cần giám sát liên tục và nhất quán.
    - Mục tiêu chính của monitoring: đảm bảo dự án đi đúng **ngân sách và lịch trình**.
    - Phải áp dụng **SCM (Software Configuration Management)** để quản lý thay đổi.
2. **Sai lầm phổ biến**
    - **Thêm nhân lực vào dự án trễ** thường khiến dự án càng trễ hơn, vì phải mất thời gian đào tạo người mới.
3. **Hoạt động cần thực hiện thường xuyên**
    - Tổ chức **project status meetings** (hàng ngày, hàng tuần, hoặc hàng tháng).
    - Thực hiện **project reviews** định kỳ và đánh giá kết quả.
    - Kiểm tra milestone có đạt như kế hoạch không.
    - So sánh ngân sách thực tế với ngân sách dự kiến.
    - Trao đổi không chính thức với nhân viên để có đánh giá chủ quan về tiến độ.
4. **Best Practices cho quản lý dự án CNTT**
    - Xác định rõ **scope** (cả những gì nằm ngoài phạm vi).
    - Định nghĩa rõ ràng **yêu cầu kinh doanh và kỹ thuật**.
    - Đặt **kỳ vọng thực tế** ngay từ đầu.
    - Dự phòng thêm thời gian và ngân sách.
    - Thiết lập **goals và KPIs** rõ ràng.
    - Chuẩn bị cho **thay đổi**, có quy trình quản lý change.
5. **Tips khi dự án đi lệch hướng**
    - Ngừng đổ lỗi, cùng nhau tìm giải pháp.
    - Quay lại cơ bản: xem lại scope, yêu cầu, trao đổi với stakeholders.
    - Giao tiếp thường xuyên và minh bạch.
    - Đảm bảo có đủ nguồn lực và tài chính.
    - Project manager phải tự đánh giá liên tục và báo cáo nếu thiếu nguồn lực.
6. **Nguyên tắc cuối cùng**
    - "Manage the process, don't let the process manage you" → quản lý quy trình, đừng để quy trình kiểm soát bạn.

📌 Ý nghĩa thực tiễn

Ví dụ: trong một **dự án phát triển ứng dụng thương mại điện tử**

- Nếu milestone “checkout system” bị trễ, thay vì thêm người mới, project manager nên rà soát lại scope, yêu cầu, và phân bổ lại nguồn lực.
- Thường xuyên so sánh chi phí server thực tế với ngân sách dự kiến để tránh vượt chi phí.
- Thiết lập KPI: thời gian phản hồi < 2 giây, tỷ lệ bug < 5% mỗi sprint.